# 本文收录于

## FiroYu/learning-vault

模型与模型应用学习库 · Agent 文献系列

———

由 FiroYu 的 Agent 整理

https://github.com/FiroYu/learning-vault

整理日期 2026-10 · 素材可溯源

# Agent 架构设计

从单 Agent 主循环到多 Agent 编排 · 中文教学材料

精读两篇原文：Anthropic《When to fold subagents into your main agent loop》

LangChain《Choosing the Right Multi-Agent Architecture》

2026 年 10 月

# 目录


前言 ..... 4

本材料的结构 ..... 4

阅读建议 ..... 4

体例说明 ..... 5

第 1 章 引言：为什么要认真谈 Agent 架构选型 ..... 6

1.1 为什么"多加一个 Agent"不总是对的 ..... 6

1.2 两篇文章讲了什么：定位与互补 ..... 7

1.3 全书学习路线图 ..... 8

第 2 章 Anthropic 精讲：何时把子 Agent 收编进主循环 ..... 10

2.1 子 Agent 的价值与三大适用条件 ..... 10

2.2 大型组织的五种典型失败模式 ..... 11

2.3 解法：单 Agent 循环 + 渐进式披露 ..... 12

2.4 目标架构：单循环设计详解 ..... 14

2.5 迁移路径：eval 先行、逐步收编 ..... 17

2.6 多团队共建单 Agent：MCP 工具、Skills 与知识库 ..... 20

2.7 官方工具箱与核心原则 ..... 23

第 3 章 LangChain 精讲：四种多 Agent 架构模式 ..... 25

3.1 前提：单 Agent 优先与两大约束 ..... 25

3.2 Subagents：集中式编排 ..... 26

3.3 Skills：渐进式披露 ..... 27

3.4 Handoffs：状态驱动转移 ..... 28

3.5 Router：并行分发与综合 ..... 30

3.6 需求-模式匹配：两张决策表 ..... 32

3.7 性能实测：三种场景对比 ..... 34

第 4 章 综合决策框架：两篇文章的合流 ..... 40

4.1 五条共识原则 ..... 40
4.2 视角差异与互补 ..... 41
4.3 架构选型决策树 ..... 42
4.4 实践检查清单 ..... 44
第 5 章 总结与思考题 ..... 46
5.1 全书要点回顾 ..... 46
5.2 思考题 ..... 47
5.3 动手实践作业 ..... 49
附录 A 术语表 ..... 52
附录 B 原文信息与延伸阅读 ..... 55
原文信息 ..... 55
章节与原文的对应关系 ..... 55
延伸阅读 ..... 55
引用与免责说明 ..... 56

# 前言

2025 年以来，"多 Agent 系统"几乎成了 Agent 领域的默认答案：仿佛不拆出几个子 Agent、不上一个编排器，就称不上在做 Agent 工程。本材料精读的两篇原文，恰好对这一潮流给出了互补的修正——一篇教你**做减法**，一篇教你**按需做加法**：

- **文章 A（Anthropic）**《Building agents: When to fold subagents into your main agent loop》：一家 Agent 工程团队复盘自己把"编排器 + 多个子 Agent"的存量系统**收编回单 Agent 主循环**的全过程——为什么收编、怎么安全地收编、收编后如何证明没改坏。
- **文章 B（LangChain）**《Choosing the Right Multi-Agent Architecture》：把"多 Agent 架构"拆成 Subagents、Skills、Handoffs、Router 四种模式，逐一给出**适用条件、性能量级与代价**，并强调其中两种本质上是"共享上下文的单 Agent 变体"。

两篇文章合起来，构成一个完整的选型坐标系：**默认从单 Agent 循环起步，能力不够先做渐进式披露，确有需要再引入多 Agent 机制；一切加减法都由端到端评测说了算。**

## 本材料的结构

| 部分 | 内容 |
|---|---|
| 第 1 章 | 两篇文章的核心论点对照，为什么它们合起来才完整 |
| 第 2 章 | 文章 A 精讲：单 Agent 循环、渐进式披露、五种失败模式、收编路径 |
| 第 3 章 | 文章 B 精讲：四种架构模式的需求表、性能场景、选型原则 |
| 第 4 章 | 综合决策框架：两篇文章的合流，附完整决策树 |
| 第 5 章 | 总结与思考题 |
| 附录 A | 术语表（中文 / 英文 / 一句话解释） |
| 附录 B | 原文信息与延伸阅读 |

全书共 8 幅插图，均为按原文内容绘制的示意图或数据重绘图，图注均标明对应章节。

## 阅读建议

- **只想建立判断力的读者**：读第 1 章 → 第 4 章的决策树 → 回头精读第 2 章 2.4 节（五种失败模式）与第 3 章 3.5 节（性能场景）。
- **正在做多 Agent 项目的读者**：重点读第 2 章的收编路径与评测护栏、第 3 章的需求表——先问"我的问题属于哪一格"，再谈架构。

- **初次接触 Agent 的读者**：按章节顺序读即可；遇到术语先查附录 A，正文首次出现时也附有英文原文。

## 体例说明

- 术语首次出现采用「中文（English）」格式，如"渐进式披露（progressive disclosure）"。
- 原文中的关键数字（如 90.2%、40%、67%）在正文中保留原值，其含义与出处见对应章节的说明。
- 引用原文观点的段落以引用块（左侧竖线）标出；其余为编者对原文的展开解释与两篇文章的交叉对照，不额外标注。

# 第 1 章 引言：为什么要认真谈 Agent 架构选型

## 1.1 为什么"多加一个 Agent"不总是对的

在绝大多数团队的工程实践中，每当智能体面临一项新的复杂能力需求，工程师的第一本能往往是："再拆一个专有 Agent 出去"。然而，**任何软件架构的引入都是有物理成本的，多 Agent 架构从来不是免费的午餐。**

在软件工程早期，微服务架构曾引发盲目的拆分热潮，最终许多团队发现分布式事务、网络延迟与跨服务调用链排查的开销远超业务本身收益。当下的 AI Agent 研发正在经历完全相同的历史周期：一旦业务流程变得复杂，开发者便本能地堆叠规划器（planner）、路由器（router）以及五花八门的垂直领域子 Agent（subagent）。然而，Anthropic 与 LangChain 近期的工业界实践研究均旗帜鲜明地反对这种"遇到问题就拆 Agent"的本能反应。

架构的成本并非抽象概念，盲目引入多 Agent 模式会在生产环境中带来四项极其沉重的直接代价：

1. **上下文传递损耗（Context Handoff Loss）**：分布式 Agent 之间依赖自然语言或固定 Schema 传递状态。就像现实世界中的"传话游戏"，协调者向下分发或子 Agent 向上汇报时，往往伴随着严重的语义压缩与细节丢失。更严重的是，多角色切换极易引发人设与口吻的割裂——某知名跨国企业曾在客服系统内堆叠了十几个专家子 Agent，结果导致对外品牌口吻无法保持统一，且各子 Agent 在边界模糊的业务场景下频现踢皮球式的死循环。
2. **端到端延迟（Latency）激增**：多 Agent 编排机制（Harness）要求请求在多个独立的模型推理调用之间串行流转。用户的一次简单交互，背后可能经历主 Agent 解析、路由判定、子 Agent 执行、主 Agent 结果汇总等多轮模型调用。Anthropic 的客户实测数据显示：**将分散的子 Agent 架构重构并收编进单 Agent 循环后，执行速度比多 Agent 编排机制快约 2 倍（2x）**。在对交互响应要求严苛的在线系统中，额外的模型往返延迟往往直接破坏了用户体验。
3. **Token 与算力成本膨胀**：每个独立的子 Agent 都需要携带自己的系统提示词（system prompt）、少样本示例（few-shot examples）以及对齐指令。多个子 Agent 并存意味着这些高昂的提示词底座在每次交互中被反复推演、重复计费。
4. **维护复杂度与组织同构（Maintenance Complexity & Conway's Law）**：在企业组织中，康威定律（Conway's Law）几乎必然显现——企业往往按照内部团队划分或问题类型各自分配一个子 Agent，使得整个 Agent 系统最终沦为企业组织架构图的机械复制品。当业务边界变动时，子 Agent 之间的职责重叠与协调冲突会导致维护复杂度呈指数级上升。

因此，**优秀的 Agent 架构设计不是看系统里塞进了多少个 Agent，而是看能否用最少的抽象层级与模型调用次数，稳定地交付预期的业务质量。**

---

## 1.2 两篇文章讲了什么：定位与互补

面对多 Agent 架构的泥潭，工业界两座灯塔——模型提供方 Anthropic 与框架构建方 LangChain，分别基于大规模落地观察给出了极具分量的解决方案。

Anthropic 在《Building agents: When to fold subagents into your main agent loop》中扮演了**"架构做减法"的布道者**。其立足于那些已经拥有庞杂子 Agent 的企业系统，提出核心主张：在多数企业应用场景下，应当果断把路由器、规划器和领域子 Agent 重新收编进单 Agent 的主循环之中。Anthropic Applied AI 团队指出，通过引入渐进式披露（progressive disclosure）设计，单 Agent 配合技能（skills）机制不仅在系统质量上优于"一个 prompt 包打天下"与"臃肿的子 Agent 编排"，而且在单任务成本和延迟上展现出压倒性优势。

LangChain 则在《Choosing the Right Multi-Agent Architecture》中扮演了**"架构做选择"的引路人**。其立足于准备构建复杂系统的开发者视角，系统性地回答了"何时必须从单 Agent 升级"以及"四种主流多 Agent 模式如何科学选型"。LangChain 明确指出，绝大多数任务的最佳起点是"单 Agent + 设计良好的工具"，只有在撞上上下文管理（专门知识超出单窗口容纳能力）与分布式开发（多团队必须独立维护各自能力边界）这两大硬性约束时，才应考虑架构升级。文章给出了涵盖子 Agent（Subagents）、技能（Skills）、交接（Handoffs）与路由器（Router）四种模式的量化性能横评：在单次请求下，集中式子 Agent 因编排层开销需消耗 4 次模型调用，而其余模式仅需 3 次；在连续对话场景中，Skills 与 Handoffs 凭借上下文复用可节省 **40%** 的调用或开销；而在跨多领域查询时，Subagents 与 Router 凭借上下文隔离与并行优势，能比 Skills 少处理 **67%** 的 token 消耗。

两篇核心文献的定位对比如下表所示：

| **比较维度** | **文章 A：Anthropic《When to fold subagents...》** | **文章 B：LangChain《Choosing the Right...》** |
|---|---|---|
| **观察视角** | 针对已被复杂多 Agent 拖累的企业系统，探讨"何时做收编与减法" | 针对面临架构选型的系统设计者，探讨"何时做升级与模式决策" |
| **核心问题** | 为什么大量子 Agent 是低效反模式？如何优雅收编？ | 什么时候单 Agent 不够用？四大多 Agent 模式各自的性能代价是什么？ |
| **核心机制/方案** | 单 Agent 主循环 + 技能（Skills）的渐进式披露 | 四种架构模式（Subagents / Skills / Handoffs / Router）的量化权衡 |

| 比较维度 | 文章 A：Anthropic《When to fold subagents...》 | 文章 B：LangChain《Choosing the Right...》 |
| --- | --- | --- |
| **量化实测数据** | • Claude Code 借助渐进式披露精简提示词超 **80%**<br>• 单循环相比多 Agent 机制速度提升约 **2 倍（2x）** | • 单次请求 Subagents 需 4 次调用，其余为 3 次<br>• 重复请求下 Skills/Handoffs 节省 **40%**<br>• 跨领域查询中 Subagents 比 Skills 少处理 **67%** token |
| **子 Agent 保留条件** | ① 上下文膨胀出现可测退化<br>② 任务需并行且上下文用完即弃<br>③ 工具选择混淆或提示词冲突严重 | 遭遇两大硬约束：<br>① 上下文硬性溢出<br>② 组织需要分布式隔离开发 |
| **适用读者** | 期望优化线上系统延迟、降低故障率与 Token 成本的中高级架构师 | 面临技术选型、需要权衡吞吐/成本/复杂度的新系统设计者与工程团队 |

这两篇文章并非相互排斥，而是构成了**现代 Agent 架构设计的互补闭环**：LangChain 的文章 B 提供了一张全局的“选型地图”，告诉你从单 Agent 起步后何时向外走、各条路径的性能开销几何；而 Anthropic 的文章 A 则是一份“收编指南”，提醒你在走得过远、系统失控时，如何通过优雅的渐进式机制做减法，把多余的实体收回主循环。

---

## 1.3 全书学习路线图

本教材旨在摒弃充斥于网络博客中的概念炒作，完全从企业真实工程交付的维度，帮助研发团队建立起严密、可量化的 Agent 架构决策思维。

全书分为五个递进模块，章节规划与学习收益如下：

### 第 1 章 引言：Agent 架构选型的核心问题

- **本章定位**：打破“多 Agent 即先进”的技术神话，确立“架构有成本”的工程基线，阐明两篇核心文献的知识互补体系。
- **学完收获**：建立对 Agent 系统延迟、Token、语义损耗的量化成本敏感度，掌握现代 Agent 架构演进的核心脉络。

### 第 2 章 架构做减法：Anthropic 单循环收编之道

- **本章定位**：深入精读文章 A，剖析单 Agent 主循环如何通过技能（Skills）与渐进式披露（progressive disclosure）实现对规划器、路由器的降维打击。
- **学完收获**：掌握 Claude Code 将系统提示词削减超过 **80%** 的渐进式工具加载实战技巧；深刻理解保留独立子 Agent 的三项硬性判定准则，学会诊断企业现有 Agent 资产中的过拟合与康威定律陷阱。

### 第 3 章 架构做选择：LangChain 四大多 Agent 模式与实测剖析

- **本章定位**：全面拆解文章 B，逐一剖析子 Agent（Subagents）、技能（Skills）、交接（Handoffs）与路由器（Router）四种模式的底层工作原理与代码形态。
- **学完收获**：深入理解三大性能基准测试背后的技术机理，掌握为何 Handoffs 在长会话中能节省 **40%** 开销、为何跨领域搜索时 Subagents 上下文隔离能节省 **67%** token 的量化取舍依据，建立精准的开销预估模型。

### 第 4 章 融会贯通：企业级统一决策框架

- **本章定位**：将文章 A 的“收编原则”与文章 B 的“选型矩阵”融为一体，输出面向企业实战落地的统一架构决策树与度量指标。
- **学完收获**：获得一份开箱即用的技术决策指南——明确从何时“单 Agent + 工具起步”，何时“渐进式技能扩展”，再到何时“升级为跨 Agent 交接或并行子 Agent”，并掌握可落地运行的回归重构流程。

### 第 5 章 总结与思考题

- **本章定位**：全书核心方法论凝练，结合工业界反模式案例，提供场景化沙盘推演与实战思考题。
- **学完收获**：通过多套典型企业级业务场景（客服协同、代码助手、多源投研分析等）的实操推演，彻底固化对不同 Agent 架构边界与折衷代价的认知掌控力。




图 1：从单 Agent 到多 Agent——Agent 架构选型全景图

# 第 2 章 Anthropic 精讲：何时把子 Agent 收编进主循环

## 2.1 子 Agent 的价值与三大适用条件

在多智能体系统设计中，子 Agent（Subagent）本质上是由顶层协调器（Coordinator）统筹调度的特化执行单元，每个子 Agent 均持有独立的上下文窗口（Context window）、专有工具集以及独立的系统指令。引入子 Agent 的核心动机，在于通过物理隔离的上下文环境降低单一提示词的认知负荷，实现特定任务的高效解耦。

在智能体架构演进的早期，子 Agent 在处理复杂长链路任务时展现出显著工程优势，其最擅长处理的场景主要集中在以下三个维度：

- **结果验证（Verification）**：在关键代码生成、大额金融交易或高风险内容发布前，由独立的验证子 Agent 以“旁观者”视角进行黑盒对抗测试与逻辑审计，避免主流程的思维定势；
- **并行任务（Parallel work）**：面对需要广泛分头检索不同信息源或并发执行非依赖任务时，分发多个子 Agent 各自建立崭新的上下文，并发获取结果后再行聚合；
- **模块化权限（Modular permissioning）**：针对涉及敏感情资或高权限执行系统的环节，将高危工具严格限定在具备鉴权围栏的特定子 Agent 内部，主循环无法直接触碰敏感接口。

尽管 Anthropic 在企业级落地实践中发现，大量原本挂载的路由器（Router）、规划器（Planner）以及领域子 Agent 完全可以收编进主 Agent 循环（Main agent loop），但子 Agent 并非应当被全面取缔。**在架构设计中，只有当系统触及以下三大客观约束时，才属于仍应坚持保留子 Agent 的有效场景**：

1. **上下文膨胀引发可度量的性能衰减**：单一 Agent 循环在长程交互中累积了过多历史信息，经过基准评测证实上下文膨胀（Context bloat）已对模型注意力和遵循指令的能力产生实质性负面影响；
2. **多任务并行且上下文阅后即焚**：系统必须并发处理多个独立分支任务，且各分支需要高度干净、无历史污染的全新上下文窗口，任务一旦完成即可将其中间过程彻底丢弃；
3. **工具选择混乱或提示词硬性冲突**：单一模型在庞杂的工具集合面前频繁出现幻觉或工具误选，或者在同一个系统提示词中存在互斥的业务规则与角色指令，导致模型难以权衡。

然而，工程现实中的子 Agent 使用现状往往与上述严谨的架构决策南辕北辙。Anthropic 应用 AI 团队在协助大量企业客户落地时，观察到一套高度程式化却极为臃肿的典型设计：前端先挂载一个意图分类器（Intent classifier），紧接着串联固定多步推理流水线（Multi-step reasoning pipeline），随后分流至各个用户分群桶（User-segment buckets），再切入各个业务域独立的推荐分支，最终在最外层套上一个复杂的多 Agent 编排器（Multi-agent orchestrator）。

产生这种臃肿架构的根源并非技术客观需要，而是**康威定律（Conway's Law）在智能体架构上的具象化投射——系统的技术架构直接沦为了企业组织架构图的机械复制品。**企业习惯于给每一个业务团队或问题类型单独分配一个专属子 Agent，把跨部门的权责壁垒原封不动地搬进了大模型的上下文拓扑中。事实证明，这些层层叠叠的控制结构所做的工作，当前的前沿大模型本身早已无需外围系统代劳；一套清晰的系统提示词（System prompt）、4 至 5 个轻量数据工具（Thin data tools）以及严谨的输出护栏（Output guardrails），就足以覆盖原本需要庞大编排矩阵才能应对的绝大多数场景。

---

## 2.2 大型组织的五种典型失败模式

当大型企业尝试将跨部门的多产品、多项业务能力整合进统一的客户交互界面时，最直观的开发路径往往是“各自为政”：各产品团队分别开发自己的专用 Agent，再由中央平台团队在顶层搭建一个编排器，通过“编排器-子 Agent 架构（Orchestrator-subagent architecture）”或“主 Agent 调用封装为 MCP 工具的子 Agent（Agents exposed as Model Context Protocol tools）”进行系统集成。这种自底向上的分治策略看似分工明确，实则迅速引发以下五种致命的失败模式（Failure modes）：

### 1. 上下文管理失控（Context management）

当任务在中央编排器与各业务子 Agent 之间流转时，架构师必须频繁裁定“哪些历史信息需要跨边界透传”。由于缺乏全局视野，开发者往往采用截断或摘要的方式向下同步状态，导致关键业务细节在来回传递中严重损耗。

### 2. 子 Agent 无序蔓延（Subagent sprawl）

由于缺乏统一标准与全局负责人，每个部门为解决局部诉求不断上线新的子 Agent。最终，企业内部充斥着数百个功能重叠、命名混乱的专用智能体，形成技术债务的泥潭。

### 3. 缝合怪提示词（The Frankenprompt）

在多团队共治的架构下，每个业务线都会以“确保本部门合规与偏好”为由，向顶层全局系统提示词中硬塞几句指令。最终，系统提示词演变为充斥着相互冲突规则、口吻（Voice）四分五裂的“缝合怪”，导致输出毫无连贯的品牌体验可言。

### 4. 无人把关的局部过度评测（Unvetted, overly localized evals）

各个业务团队只针对自己负责的垂直子任务编写基准评测，并针对孤立指标实施过度优化。全局缺乏端到端的评测集，管理层看到的评测分数高度亮眼，但这些被过度局部化的指标掩盖了端到端体验断崖式下跌的真相。

### 5. 系统端到端延迟剧增（Latency）

多 Agent 之间每一次调度都包含一次甚至多次额外的模型推理与网络交互。复杂的编排调用链路层层累加，最终对终端用户体验造成毁灭性打击。**在 Anthropic 针对某客户的实际工程基准测试中，将多 Agent 编排线收编为单一 Agent 循环后，系统的端到端执行速度提升了约 2 倍（2x faster runtime）。**

| 失败模式 | 核心成因 | 典型工程后果 |
| --- | --- | --- |
| **上下文管理失控** *(Context management)* | 缺乏共享上下文，跨 Agent 边界的状态传递依赖人为截断与摘要 | 关键细节在层层转述中丢失，导致模型做出错误假设与幻觉输出 |
| **子 Agent 无序蔓延** *(Subagent sprawl)* | 各团队自建自发，缺乏企业级准入标准与全局生命周期管理 | 系统沉淀数百个功能重合的子 Agent，无团队愿意且能够全盘维护 |
| **缝合怪提示词** *(The Frankenprompt)* | 跨部门政治妥协，各业务线持续向全局系统提示词追加私有指令 | 指令集充斥内在逻辑冲突，模型语气撕裂，品牌风格完全失控 |
| **局部过度评测** *(Localized evals)* | 缺乏全链路质量把控，各团队仅考核自身负责的狭窄子步骤 | 局部指标高企而端到端体验低下，管理层被失真评测误导决策 |
| **延迟剧增** *(Latency)* | 编排器与子 Agent 之间过多的往返调度与重复推理调用 | 用户等待时间过长；实测单循环运行效率是多 Agent 架构的约 2 倍 |




图 2：架构复制组织架构图——Conway 定律下的失败模式

## 2.3 解法：单 Agent 循环 + 渐进式披露

解决上述架构泥潭的根本手段，是将路由、规划与垂直领域子 Agent 重新并入**单 Agent 循环（Single agent loop）**，并结合**渐进式披露（Progressive disclosure）**模式来治理复杂度。

所谓渐进式披露，其核心理念是打破“启动即载入全部依赖”的传统做法，将能力与上下文的呈现拆分为两阶段：

- **通过技能（Skills）渐进式披露上下文**：初始状态下，模型仅能看到各个技能的一两句简短描述。只有当主循环根据用户需求明确判定需要激活某项技能时，该技能详尽的操作规程、业务规范与参考上下文才会被完整拉取进主上下文；
- **通过工具搜索（Tool search）渐进式披露工具**：系统不再把成百上千个 API 的完整 JSON Schema 一次性注册到模型的上下文窗口中，而是仅向模型提供工具索引摘要。主循环在执行中动态检索相关工具，按需挂载完整定义。

这一架构能够取代复杂多 Agent 设计的底层逻辑建立在以下三大支柱之上：

1. **共享上下文产出更优质的答案（Shared context produces better answers）**：对于需要较长执行周期的任务，让全流程的上下文直接沉淀在同一个主上下文窗口中具有极高价值。模型能够清晰观察到前期每一步工具调用的原始返回与逻辑推演，并将其无缝关联，从而产出远比子 Agent 返回的干瘪摘要更丰富、更有深度的洞察。同时，这意味着更少的独立上下文窗口，以及**彻底规避了信息在各子系统间交接传递带来的有损流转（Lossy handoffs）。**
2. **渐进式披露有效防止上下文膨胀（Progressive disclosure prevents context bloat）**：该技术让团队无需一次性将所有业务细节硬塞进臃肿的系统提示词，便能使单一 Agent 具备跨领域的广泛执行力。这从根本上拆除了过去“不得不拆分子 Agent 以节省上下文”的技术借口。**Anthropic 在适配更高级模型时，将 Claude Code 的系统提示词体积大幅削减了 80% 以上，其中绝大部分删减归功于技能与工具的渐进式披露机制。**
3. **模型自身能力呈代际飞跃（Models are more capable）**：从 Opus 4.5 开始，大语言模型在单一循环内直接掌控与调度复杂系统的能力，已经大幅超出大多数工程团队的预期。诸如 Opus 5.5 和 Fable 5.1 等前沿模型甚至原生锁定了思考过程（Thinking cannot be turned off），新一代模型在推理过程中会自发叙述当前进度并在前行中动态检查校验结果，这直接让外围工程脚手架（Harness）中的许多控制模块变得完全多余。更为关键的是，更简单的单循环架构极大降低了向下一代大模型平滑迁移的摩擦力；在过往的实际迁移支持中，恰恰是那些僵化固定的多 Agent 外壳，阻碍了能力更强的新模型接管更多复杂工作。

推行单 Agent 循环绝不意味着降低对架构严谨性的要求，相反，它需要极其坚决的**工程纪律（Engineering discipline）**：必须严格精细化管理系统提示词；确保技能和工具的简短描述具备极高区分度，以便模型在正确的时机准确触发加载；同时必须对上下文窗口进行主动运维，包括有条理地管理记忆（Memory）与过程笔记（Note-taking）。

在组织权责分配上，必须由一个**人数精干的中央团队（Small central team）独家持有端到端质量与全局系统提示词。**任何团队提交的新工具定义或新评测项，在正式合并发布前必须经过该中央团队的严格审查与准入（Vetting）。保持中央团队的小型化，是消除大规模组织内沟通内耗、维持系统快速高频迭代的唯一可行路径。

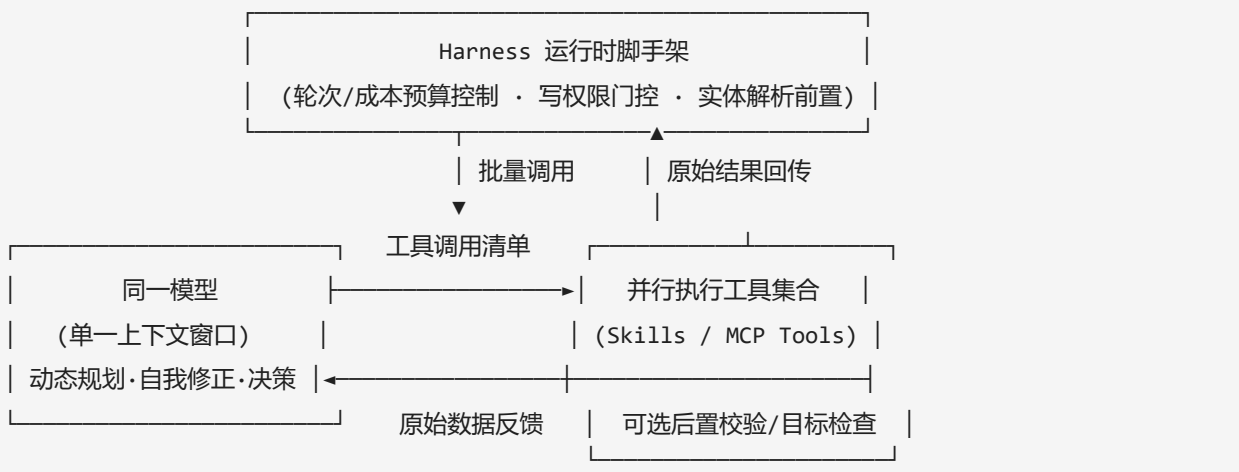
如果团队在详尽评估后仍然决定采用多 Agent 架构路线，则必须遵循两项核心防错守则：

- **从最简模式起步**：优先从生成者-验证者（Generator-Verifier）或简单的编排者-子 Agent（Orchestrator-Subagent）等最简拓扑开始构建，切忌盲目上马复杂网络；
- **严格按上下文切分边界（Dividing by context）**：给每个子 Agent 划分工作时，必须确保该子任务对系统其余部分的上下文依赖极低，且能够向主循环交付紧凑、自包含的交付物（例如相互独立的调研路径、接口边界清晰的解耦组件，或是独立的黑盒验证）。**如果错误地按照业务问题类型或组织部门来生硬切分 Agent，系统不可避免地会陷入“传话游戏（Telephone game）”的陷阱——上下文信息在智能体之间来回倒手传递，每一次转手交接都会导致信息保真度严重退化。**

## 2.4 目标架构：单循环设计详解

在智能体工程实践中，许多团队最初倾向于搭建层级分明的多智能体编排网络。然而，Anthropic 在生产实践中发现，将散落的子智能体收编并收敛为一个内聚的单循环架构（Single Loop Design），往往能获得更高的执行成功率与更低的工程复杂度。

所谓单循环设计，是指**由单一模型在单一上下文窗口（Context Window）中运行开放式的工具循环。**外部的宿主运行时脚手架（Harness）负责调度执行，并提供严格的安全边界与资源控制。



该目标架构具体包含以下五大核心要素：

### 1. 单模型与统一上下文窗口，外加 Harness 边界设限

系统不再为每个细分子任务派生独立的微型智能体，而是让同一个基础模型在同一个上下文窗口中维系完整的思考状态。为了防止开放式循环陷入死循环或产生不可控支出，**Harness 必须在外周施加硬性约束**：设置轮次上限预算（Turn Budget）、成本花费预算（Cost Budget），并设立关键写操作的权限验证门控（Write Gates）。模型负责在规则内自由探索，Harness 负责死守安全底线。

### 2. 批量并行工具调用与原始结果回传

在循环的每一个轮次中，模型可以一次性派发一批工具调用请求。Harness 接收后以并发形式并行执行这组调用，并**将未加损耗的原始执行结果（Raw Results）直接回传给同一个模型**。模型根据收集到的全量真实反馈决定下一步行动：是发起下一批并发工具调用、向用户提出追问，还是直接给出最终解答。

### 3. 自主终止机制与后置验证

循环的终止条件十分清晰：当模型判断任务已达成并输出最终回复时终止，或在触及 Harness 设定的预算红线时被动截断。此外，系统可以在模型输出最终答案之后，挂载一个可选的目标检查器或验证器（Verifier Check），对交付质量做最后的确定性审查。

### 4. 动态计划制定与就地修订：计划者即执行者

这是单循环架构与传统"编排器 + 专家 Agent（Orchestrator and Specialist Agents）"范式的本质区别。在传统的编排架构中，编排器负责制定计划，随后像流水线工长一样把子任务派发给各个专家智能体。专家并不理解全局意图，编排器也难以感知底层工具返回的细粒度细节。

而在单循环设计中，**模型在第一轮交互中直接将执行计划写入当前上下文，并随着后续工具调用结果的返回动态修正计划。**由于**计划制定者本身就是执行者（The model that writes the plan is also the one carrying it out）**，当工具报错或返回意外数据时，模型能够在执行中途（Mid-turn）就地更改计划走向，无需经历多层级的汇报与重新派发。

### 5. 上下文由智能体自主管理

在过去的多层架构中，系统往往需要编写胶水代码，根据一套预定义的规则目录去动态改写编排器的计划。单循环设计彻底废弃了这种代码重写步骤，转而让智能体全权自主管理其上下文：

- **业务流程与调用顺序下沉至技能（Skills）**：领域规范、最佳实践以及建议的执行顺序不再写死在代码控制流中，而是封装进按需加载的 Skills 提示词模块。

- **硬性约束与即时检索下沉至工具（Tools）**：诸如严格的数据校验逻辑、人工批准门控（Approval Gates），以及需要中途发起的记忆检索（Memory Lookup），统一封装为确定性的工具接口。
- **低成本通用逻辑保留在代码层**：对于成本极低且几乎在任何场景下都有用的步骤（例如确定性的实体解析/Entity Resolution），以及工具内部的机械性处理（载荷构造、数据流解析、澄清后的任务恢复），依然交由确定性代码高效处理。

| 架构维度 | “编排器 + 专家 Agent” 架构 | 单循环架构（Single Loop） |
|---|---|---|
| **状态维系** | 状态割裂，各专家拥有孤立上下文，靠摘要回传 | **全量状态保留在单一上下文窗口中** |
| **计划调整** | 计划者不执行；遇到异常需层层汇报，重写逻辑繁琐 | **计划者即执行者，感知原始数据，支持中途就地改计划** |
| **调用损耗** | 嵌套模型循环带来高昂延迟，跨 Agent 汇报信息有损 | **单层循环，Harness 批量并行调用工具，低延迟、零信息衰减** |
| **流程控制** | 硬编码胶水代码与复杂的规则调度引擎 | **流程沉淀为 Skills，约束与检索沉淀为 Tools** |

单循环设计高度契合了智能体架构演进的终极目标——**构建一个能够深刻理解用户高层意图、并代表用户直接行动的高级智能体**。将智能体在解决问题过程中学到的一切信息、踩过的坑与即时数据沉淀在统一的上下文中，是实现高阶自主推理的物理基石。




图 3：单 Agent 主循环设计——计划、并行工具调用与自我修正

## 2.5 迁移路径：eval 先行、逐步收编

将生产环境中错综复杂的子智能体系统重构成单循环架构，绝不能凭直觉一蹴而就。缺乏基准测试的重构等同于裸奔，极易在降低延迟的同时悄然破坏核心业务逻辑。团队必须确立**评测（Eval）先行、渐进收编**的演进策略。

### 建立面向任务完成度的评测体系

收编重构的第一步不是改写 Prompt 或合并代码，而是建立一套能够量化评估“任务是否真正完成”的端到端评测集（Task-completion Evals）。一套行之有效的评测资产组织架构通常包含两个层级：

1. **中央团队持有共享的端到端评测集**：覆盖跨领域、全流程的业务主链路。团队无需一开始就追求海量数据，**仅需 10 条高质量的黄金样本（Golden Examples）就足以启动第一阶段的收编工作。**
2. **各技能团队附带专属的小型评测集**：每一个被收编的业务技能模块（Skill），都由其负责团队自带一组小而精的局部评测用例，用于验证该特定领域规则的有效性。

在评测指标的选取上，团队需要使用**评分准则（Rubric）**对智能体的最终回复质量进行打分。需要注意的是，评测指标必须随着架构演进动态切换：

- **保留编排器阶段**：重点追踪**路由准确率（Routing Accuracy）**，即编排器是否将任务准确分发给了对应的子智能体。
- **收编进单循环阶段**：将指标切换为**行为调用准确率**，即检验主智能体是否在正确的阶段加载了正确的 Skill，或者是否正确调用了对应的 Tool。

在企业级真实生产环境中，常常面临底层内部系统难以无损回放、真实用户交互难以模拟的工程痛点。对此，Anthropic 内部验证有效的方案是：**在生产系统的关键重要事件上打上时间戳（Timestamps）**，以此精准捕获线上数据快照与状态流，后续便可依托时间戳直接回放真实场景，执行高保真的定向基准评测。

## 三条渐进式收编路线

有了评测体系保驾护航后，便可通过以下三条路线逐步瓦解并合并子智能体：




***路线一：合并紧密依赖的领域智能体（Combining Domain Agents）***

在执行首次合并之前，必须先搭建好对应场景的小型端到端评测。合并的推进原则是：**依据共享上下文展开收编，且优先从生产环境中调用频次最高的子智能体入手。**

具体而言，应该将那些**输出存在链式依赖（即 A 的输出紧接着作为 B 的输入）、频繁读写相同底层数据，或者在实际流量中经常被成对调用的专家智能体合并为一个领域智能体。**例如，若生产指标显示“订单查询”后有极高比例紧接着调用“物流跟踪”，这两个紧密耦合的专家就应当首批合并，避免无谓的跨 Agent 状态同步。

***路线二：将子智能体降级为工具调用（Subagents Called as Tools）***

如果某些专家子智能体暂时无法完全拆解，一种稳妥的过渡方案是将其作为主智能体的工具进行调用。

在这种模式下，**主智能体始终保留与用户的直接对话权。**行业中常见的“完全交接（Full Handoff，即将用户对话控制权完全移交给子智能体）”应当受到极其严格的限制，**只有在子智能体确实必须直接与用**

**户进行多轮深入澄清的极少数场景下，才允许进行会话移交。**其余情况下，专家智能体都应作为主循环下的工具存在。

这种过渡方案具有明显的工程收益：

1. **无需彻底重写业务逻辑**：专家智能体的代码与逻辑无需推倒重来即可接入。
2. **便于建立评估基线（Baseline）**：它能为主智能体在任务完成度评测中快速提供一个对照基准。
3. **支持契约化与交叉验证**：团队可以为主智能体与子智能体之间定义标准化的共享响应模式（Shared Response Schema），主智能体还能拿着工具返回的结果与真实数据对照验证，并在最终汇总输出前对冗余上下文进行剪裁。

**但必须清醒认识到，路线二仅仅是过渡形态，仍残留着三大固有缺陷**：

- 每一个专家依然保留独立的提示词、上下文、维护团队与评测集，工程割裂问题未完全解决；
- 每次工具调用返回给主智能体的依然是**二次加工后的有损摘要（Lossy Summary）**，底层细节缺失；
- 每次调用本质上是在主循环中嵌套了一个子模型循环，**导致系统的整体调用延迟（Latency）成倍攀升。**

***路线三：彻底将子智能体折叠为 Skills 与 MCP 工具***

终极形态是彻底拆解子智能体：将其专有提示词与流程知识改写为 Skills，将其底层的 API 调用改造为符合模型上下文协议（Model Context Protocol, MCP）的标准工具。

在实施折叠时，必须采取"逐个击破"的方式：每次只拆解一个子智能体，并在开启与移除该子智能体两种状态下对比评测集表现，据此决定去留。

> [!WARNING] **绝对不能只看评测的总分，必须深入审视具体是哪些任务发生了"结果翻转（Flipped Tasks）"。**

在 Anthropic 的真实客服智能体测试案例中，团队将一个子智能体折叠进主循环后，整体系统的响应延迟显著降低，综合得分看似维持平稳；但深入排查翻转用例时发现，**一个原本能够稳定通过的业务流程被彻底破坏了**——主智能体因为拥有了底层的订单查询工具，便在未向物流专家确认实际发货日期的情况下，擅自直接向用户作出了不准确的回答。

这一经典教训给工程师敲响了两记警钟：

1. **改动前必须确保评测覆盖全面**：评测集必须完整覆盖在生产环境中一起运行的每一条智能体调用链，以及业务中不可逾越的每一条红线规则。

2. **每个评测用例必须执行多次**：大模型的非确定性意味着隐蔽缺陷具有间歇性。在上述案例中，该违规跳过步骤的破坏行为在两次运行中仅暴露了一次（Only showed up in one run out of two）。如果单次测试碰巧通过，缺陷就会长驱直入流入生产。

| 路线方案 | 实施方式 | 核心优势 | 残留问题 / 风险点 | 评测关注重点 |
|---|---|---|---|---|
| **路线一：合并领域 Agent** | 合并链式调用、共享数据的高频专家 | 减少编排层级，消除冗余上下文传递 | 仍存在局部子 Agent 边界与管理成本 | 链式任务整体完成率、重叠上下文冗余度 |
| **路线二：子 Agent 作为工具** | 主 Agent 控场，子 Agent 作为普通工具被调用 | 无需重写旧模块；快速产出 baseline；统一对话入口 | 嵌套模型循环增加延迟；每次调用返回有损摘要 | 模式（Schema）校验率、主 Agent 交叉校验成功率 |
| **路线三：折叠为 Skills+MCP** | 提示词转为 Skills，API 转为标准 MCP 工具 | 彻底清除嵌套延迟；原始数据零损耗；自主灵活规划 | 提示词干扰可能导致模型绕过特定专家流程 | **翻转任务（Flipped Tasks）**；单用例多次运行稳定性 |

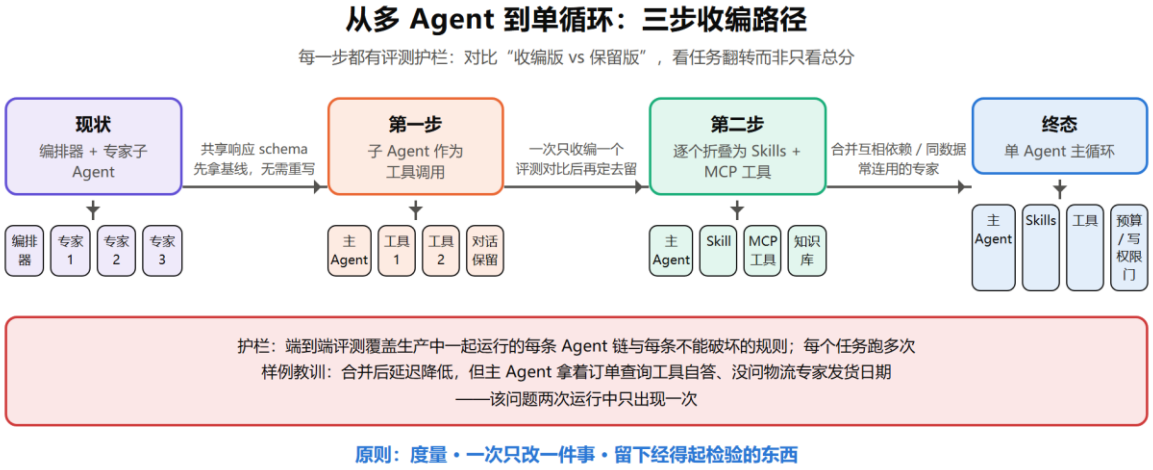



图 4：从多 Agent 到单循环的三步收编路径

## 2.6 多团队共建单 Agent： MCP 工具、Skills 与知识库

多团队协同并不意味着每个团队都必须各自部署一套独立的智能体系统；更优的工程实践是将分散的领域推理能力解耦，统一沉淀为可插拔的扩展资产。

在单智能体循环（Single Agent Loop）的组织架构中，**各个业务团队的角色应从“拥有独立的智能体（Agent）”转变为“拥有高标准的技能（Skills）与工具（Tools）”**。系统的顶层所有权则收敛至一个精干的**中央小团队（Central Team）**，由其牢牢把控核心系统提示词（System Prompt）与端到端质量（End-to-End Quality），并在任何新技能、新工具和评测集正式上线前完成严格审核把关。

业务领域的知识与推理逻辑主要通过三类载体向主循环注入：模型上下文协议工具（Model Context Protocol tools，简称 MCP 工具）、技能（Skills）以及知识库（Knowledge Bases）。为了让系统保持清晰高效，我们需要明确三者的边界与适用场景。

| 资产类型 | 核心定位与适用场景 | 内容形态与生命周期 | 典型示例 |
| --- | --- | --- | --- |
| **MCP 工具（Tools）** | **执行动作、读取实时数据、返回固定格式结果** | 具备确定性输入输出的可执行接口，随 API 变更维护 | 数据库实时查询、下单接口、工单状态变更、外部 API 调用 |
| **技能（Skills）** | **沉淀工作流规则、操作步骤与专家级主观判断** | 结构化的工作指引与提示词逻辑，随业务经验迭代 | 复杂退换货仲裁规则、多表联合诊断分析指引、代码合规审计规程 |
| **知识库（Knowledge Bases）** | **仅供偶尔检索查阅的非结构化静态参考资料** | 放置在搜索/查询工具之后或作为技能内部的附带文件 | 公司报销政策、产品技术规格白皮书、业务核心指标口径定义 |

### 1. MCP 工具（MCP Tools）的设计与工程边界

当主循环需要产生外部副作用、从外部系统拉取实时状态或要求以强类型固定结构返回数据时，工具是最合适的手段。甚至在多智能体系统向单循环收拢的过渡期，你还可以把尚未收编的专家子智能体暂时包装为一个工具供主循环直接调用。在设计工具集时，必须遵循以下工程原则：

- **按用户意图分组，拒绝按 API 机械映射**：严禁把后端的每个 RESTful API 都机械映射为一个工具。官方经验表明，**一旦注入系统提示词中的工具定义数量超过 30–50 个，大语言模型的工具选择能力就会显著退化。**因此，必须依据高层级的用户意图聚合工具功能，并辅以工具搜索与延迟加载（Tool Search with Deferred Loading）技术，按需激活。
- **精简硬性前置条件，移入工具内部校验**：尽量避免在系统提示词中堆砌复杂的调用链依赖（例如硬性规定“调用工具 Y 之前必须先调用工具 X”）。更稳健的做法是把必要的前置检查下沉到工具内部：当工具 Y 被触发时，由其自检是否存在 X 的产出；若缺失，则直接向模型返回包含可执行具体指令的明确错误提示（如：“缺少前置参数 token_id，请先调用工具 X 获取该凭证”），模型接收后便能即时自主纠偏。
- **权限校验逻辑内聚于工具**：按用户维度做权限控制（Per-user permissions）同样应当贯彻上述思路。每个工具直接核验当前调用用户的身份与权限，并在鉴权失败时返回模型可理解的通俗拒绝信息（Plain Refusal）。这种设计使得**一个主智能体循环无需为每个权限等级单独复制一套 Agent，即可安全服务具备不同访问权限的用户。**

### 2. 技能（Skills）：领域工作流的规范化资产

当智能体需要掌握“某团队是如何思考与干活”的经验（涵盖标准作业程序、业务规则以及主观裁量权）时，技能是最佳容器。这些内容恰恰是传统多智能体架构中沉淀在各个专家子智能体提示词里的核心智慧。

为了避免技能库因缺乏治理而无限膨胀（Skill Sprawl），工程团队应将技能的维护职责交还给撰写技能的业务负责人（Directly Responsible Individual，简称 DRI），并借助官方工具实现自动化审查与精简：

- **/skill-creator**：用于辅助工程师起草格式规范的新技能，同时提供自动化审计功能，用于修剪冗余描述、消除重复内容以及重构已有技能的逻辑脉络。
- **/skill-doctor**：深入剖析技能库中各项技能所消耗的 Token 成本与实际调用频次，辅助团队主动清理死重（Dead Weight）和利用率极低的僵尸技能。
- **DRI 全生命周期负责**：每个业务团队的 DRI 必须对其拥有的技能负责，确保每一项技能定义精良、与其它团队资产互不重合，且上线时必须自带配套的基准评测。

### 3. 知识库（Knowledge Bases）：按需检索的静态基石

知识库专用于容纳智能体在任务中**偶尔需要翻阅的静态参考资料**（例如法律合规条款、财务报销政策、业务指标统一定义以及硬件产品规格）。

在架构设计上，**严禁使用硬编码程序预先检索大量文本并机械拼接到主 Prompt 中。**正确的做法是将事实与参考材料置于搜索工具或按键查询工具（Lookup Tool）之后，由模型自主评估后发起检索；或者将其作为局部参考文件，存放在特定技能的目录下，随技能动态加载。

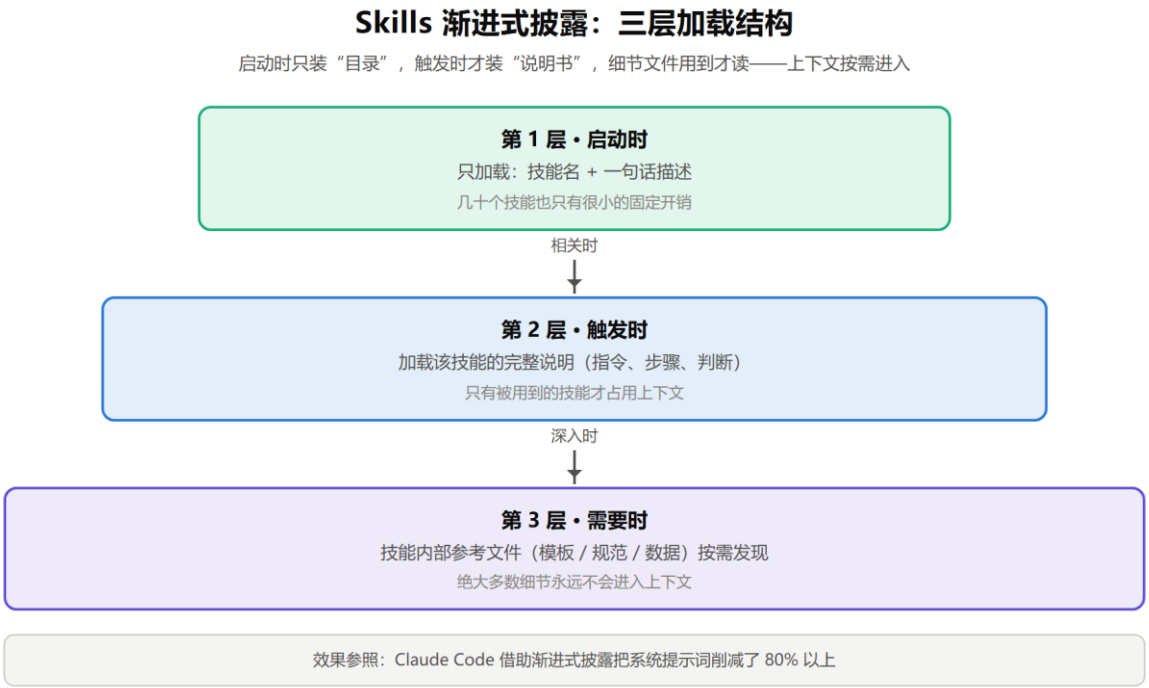



图 5：Skills 渐进式披露的三层加载结构

## 2.7 官方工具箱与核心原则

平滑地从多子智能体架构过渡到单循环并非依靠主观臆断，而是建立在科学的度量与持续集成之上。官方提供了一套完整的迁移工具包（Single-agent-loop-kit），将四个核心的 Claude API 技能串联成标准化的引导式工作流，并附带了一套可在模拟环境先行验证的演示系统（Demo）。

### 1. 四大 Claude API 核心技能

这四个自动化技能构成了智能体渐进式重构的工程闭环，覆盖了从基准建立、提示词瘦身到性能调优的全生命周期：

- **/claude-api build-eval（构建评测集）**：辅助中央团队快速建立全局的端到端评测集（End-to-End Evals），同时也支持各业务团队 DRI 为自己提交的独立技能快速构建轻量级评测基准，构筑质量底线。
- **/claude-api prompt-audit（提示词审计）**：专门排查提示词、技能描述与工具 Schema 中针对旧版模型特性编写的陈旧规则（这些补丁往往导致新型模型运行僵化、响应变慢）。该工具能以代码差量（Diff）的形式输出精确的优化补丁，中央团队和各团队 DRI 均可定期运行。
- **/claude-api hillclimb（爬坡迭代优化）**：以已有评测集为基准，对应用进行自动化闭环调优。它在**“收编版（Folded In）与保留版（Kept）的对比验证”**中发挥核心作用：当你将某个子智能体降级为技能加工具后，运行 hillclimb 持续迭代优化技能说明与工具描述，直观对比收编后的单循环能否在得分上持平甚至超越原本独立的子智能体。
- **/claude-api cost-optimize（成本与延迟优化）**：全面剖析系统调用的 Token 与时间开销分布，通过单变量逐项排查，针对提示词缓存（Prompt Caching）、剪裁冗余输入输出 Token、调整推理努力程度（Effort）以及更具性价比的模型选择提供具体的成本压降建议。

### 2. 演进的核心原则：度量、单变量、留存检验

随着基础大模型的推理能力持续攀升以及智能体单次运行上下文窗口的扩容，原本需要独立子智能体才能胜任的复杂任务，绝大多数都能被主循环凭借完善的技能与工具稳稳接管。

然而，架构的重构绝不应追求一步到位的大幅重写，工程演化的最高准则始终是：**度量（Measure）、一次只改一件事（Change one thing at a time）、留下经得起检验的东西（Keep what holds up）**。

在实际项目落地中，推荐遵循以下循环渐进策略：

1. **建立基线**：首先使用 `build-eval` 建立一个小型且敏锐的端到端评测基准。
2. **单点收编**：挑选一个边缘的专家子智能体，将其上下文抽取为特定技能（Skill），将其动作抽取为 MCP 工具（Tool），收纳进主循环中。

3. **对照检验**：借助 `hillclimb` 跑分对比收编版与保留子智能体版的指标。**唯有当收编版本的得分表现明确劣于原有多智能体系统时，才破例将其保留为独立的子智能体**；否则，一律果断收编进主循环。
4. **循环往复**：在稳固收益后，再推进下一个子智能体的收编，直到整个架构精炼高效。

# 第 3 章 LangChain 精讲：四种多 Agent 架构模式

## 3.1 前提：单 Agent 优先与两大约束

在探讨复杂的分布式架构之前，工程师必须先确立一个核心工程原则：**在绝大多数业务场景下，“单智能体（Single Agent）+ 精心设计的工具（Well-designed Tools）”始终是架构设计的首选起点。**

很多开发者在面对复杂业务时，容易陷入过早引入复杂系统的陷阱。然而，单智能体架构具有无可替代的工程优势：**它的构建门槛更低、运行时因果链条更容易被推理（Reason about），且在出现异常时具备更清晰的调试（Debug）路径。** 因此，在系统演进的初期，优先将精力投入到高质量工具的封装与单体提示词（Prompt）的打磨上，是投入产出比最高的策略。

---

### 架构演进的拐点：两大约束的浮现

单智能体并非万能。随着业务系统的规模化扩展，研发团队往往会面临一个共同的挑战：系统内衍生出了大量分散而庞杂的智能体能力，而产品目标却需要将这些能力聚合成一个连贯统一的对外交互界面。

当需要集成的特性与能力数量急剧膨胀时，单体架构就会撞上两堵坚硬的物理之墙：**上下文管理（Context Management）**与**分布式开发（Distributed Development）**。

| 约束维度 | 核心痛点 | 物理极限与现实差距 | 单体架构的局限 |
| --- | --- | --- | --- |
| **上下文管理** *(Context Management)* | 专门化知识容量超载 | 若上下文窗口无限且网络延迟为零，所有信息皆可预先全量载入；但现实中资源与延迟均受限 | 无法将所有垂直领域的专业知识塞进单个提示词中，系统必须具备动态、选择性呈现信息的能力 |
| **分布式开发** *(Distributed Development)* | 跨组织协作边界模糊 | 各业务能力通常由不同团队独立开发与维护，需要明确的权限边界与所有权归属 | 单体庞大的系统提示词（Monolithic Prompt）难以跨团队协同维护，极易因一人修改引发全局风险 |

这两项约束直接定义了单智能体架构的能力边界。**只有当你的系统需要管理海量领域知识、需要跨多团队协作，或者面临真正复杂的任务场景时，多智能体架构（Multi-Agent Architecture）才会成为理性的正确选择。**

---

### 实证支撑：独立上下文与并行推理的威力

多智能体架构之所以能在克服上述约束的同时带来性能跃迁，其底层机制在于**解耦**与**并行**。

在 Anthropic 的一项前沿多智能体系统研究中，研究团队对比了单智能体与多智能体在复杂研究任务中的表现：

- **系统架构设计**：采用 Claude Opus 4 担任主智能体（Lead Agent），负责全局规划与任务调度；配合多个 Claude Sonnet 4 作为子智能体（Subagents），分别承接具体的垂直子任务。
- **评测结果**：在内部研究评测集上，该多智能体架构的整体表现超越了单一的 Claude Opus 4 智能体达 **90.2%** 之多。

这一巨大性能差距的根本原因，正在于多智能体架构对两大约束的有效化解：系统将庞大的工作负载拆分给拥有**独立上下文窗口（Context Window）**的多个智能体执行，从而释放了单智能体所无法实现的**并行推理（Parallel Reasoning）**能力。

因此，多智能体不是盲目堆砌角色的炫技，而是面向上下文极限与组织协作复杂度的工程解法。在后续章节中，我们将基于这一前提，系统解析 LangChain 总结的四种核心多智能体设计模式。

## 3.2 Subagents：集中式编排

### 概念定义与核心机制

子智能体（Subagents）模式是一种典型的集中式编排（Centralized Orchestration）架构，其核心由主管智能体（Supervisor Agent）通过将专门化的子智能体当作工具（Tools）进行调度调用。在这一模式中，**主智能体（Main Agent）负责自始至终维护对话上下文，而专门化的子智能体则始终保持无状态（Stateless），这为系统带来了极强的上下文隔离（Context Isolation）。**

### 工作原理

子智能体模式由主智能体统一掌控全局交互与数据流向，具体体现在以下几个机制：

1. **决策与路由控制**：所有的路由逻辑统一穿过主智能体，由主智能体全权决定何时调用哪个子智能体、为其传递何种输入内容，以及后续如何归纳与合并产出的结果。
2. **状态解耦与隔离**：各个子智能体不记录过往的任何交互历史，仅针对主智能体即时传入的单一上下文执行垂直任务，有效避免了不同功能模块间上下文的互相污染。
3. **支持并行分派**：由于调度权完全集中在主管智能体手中，系统能够面向不同任务需求并行调用（Invoke in Parallel）多个子智能体，显著提升执行效率。

### 最适用场景与工程示例

该架构**最适用于拥有多个独立业务领域、需要强集中化工作流控制，且子智能体无需直接与用户产生对话交互的复杂应用场景。**

典型的工程案例包括：

- **多领域个人助理**：统一协调并调度日历（Calendar）、电子邮件（Email）与客户关系管理系统（CRM）等独立业务工具的协同运作。
- **深度研究系统**：主控系统根据研究课题，将不同垂直方向的研究任务分派给专门的领域专家子智能体，再集中汇总结论。

### 关键权衡与架构代价

子智能体模式的关键代价在于：**每次交互都会增加一次额外的模型调用（Model Call），因为所有子智能体的执行结果都必须回流至主智能体进行综合整理。**

**这种额外的调用开销换来了系统强大的集中式控制力和可靠的上下文隔离，但同时不可避免地付出了响应延迟（Latency）与令牌（Tokens）消耗的代价。**

工程落地方面，对于希望以极简配置搭建该架构的开发者，LangChain 生态中的 Deep Agents 提供了开箱即用的封装实现，仅需寥寥数行代码便可快速集成子智能体。

---

## 3.3 Skills：渐进式披露

### 概念定义与核心机制

技能（Skills）模式是指智能体按需动态加载专门的提示词（Prompts）与特定领域知识，其本质是**对智能体自身能力的渐进式披露（Progressive Disclosure）。**

虽然从技术实现层面来看，技能架构依然运行在单一智能体（Single Agent）之上，但它允许该智能体在运行期动态切换并采纳专门的“人格（Personas）”。这种机制为系统赋予了与传统多智能体架构高度相似的优势——例如支持分布式模块化开发、实现细粒度的上下文控制，同时又避免了多实例运维的沉重负担。正因如此，LangChain 团队（或许略带争议地）将技能模式定义为一种**准多智能体架构（Quasi-Multi-Agent Architecture）。**

### 工作原理

技能通常是以提示词为核心的专业化能力包，在工程实现上被组织为包含指令（Instructions）、脚本（Scripts）与资源（Resources）的文件目录结构。它的渐进式披露主要分为三层：

1. **系统启动层（第一层）**：在系统刚启动初始化时，智能体仅获知各个技能的名称及其摘要描述，将常驻上下文的消耗降至最低。
2. **技能激活层（第二层）**：只有当特定技能与当前的交互任务产生关联时，智能体才会按需加载该技能完整的上下文信息。
3. **细节发现层（第三层）**：存放在技能目录内部的补充文件提供了第三层级的详细信息，智能体仅在执行过程中实际产生需求时，才会进一步动态检索并发现它们。

### 最适用场景与工程示例

技能架构**最适用于拥有众多潜在专业化方向的单一智能体系统、能力之间无须实施刚性约束隔离的场景，以及由不同团队分别独立维护各自特定技能的协同开发环境。**

典型的工程案例包括：

- **代码编写智能体（Coding Agents）**：根据具体项目所涉及的框架、技术栈或语法规范，动态载入对应的语言与工程技能包。
- **创意助手（Creative Assistants）**：在文案撰写、剧本构思、视觉创意等差异化任务之间灵活切换专属技能。

### 关键权衡与架构代价

技能模式在整个生命周期中保持了系统的轻量性，并保障了用户与智能体之间始终保持直接交互。然而其**关键代价在于：随着技能在对话交互中被陆续按需加载，技能上下文会在对话历史（Conversation History）中不断累积，从而导致后续轮次调用中出现令牌膨胀（Token Bloat）。**

## 3.4 Handoffs：状态驱动转移

交接模式（Handoffs）是一种去中心化的状态驱动协作架构，其核心动机在于让系统的控制权跟随对话上下文的发展而自然交织与流转。在该模式下，系统不再依赖固定中心调控，而是让当前处于活跃状态的活跃智能体（Active Agent）随对话语境动态改变，每个智能体都具备通过工具调用（Tool Calling）将处理权转移给其他智能体的自主能力。

### 工作原理与状态演进

在交接模式中，多智能体之间的协作本质上是一个由状态机驱动的流转过程。

当某个活跃智能体在交互中判定当前任务需要移交，或者已达成阶段性目标时，它会触发一个专门的交接工具。调用该工具的操作会实时更新系统的共享状态，进而决定下一个被激活的智能体是谁。具体在工程实现层面，这种“交接”通常包含两种形态：

1. **实体切换**：直接将执行权移交给另一个完全不同的独立智能体实例；
2. **逻辑重塑**：不更换底层智能体实例，而是直接热替换当前智能体的系统提示词（System Prompt）与可用工具集合。

该模式最显著的底层特性在于：**系统的状态能够在跨对话轮次（Conversation Turns）中持续存活，从而为多阶段的顺序工作流（Sequential Workflows）提供坚实支撑。**每一次转移不仅传递了控制权，也延续了对话生命周期内的关键业务上下文。

## 典型适用场景与现实类比

交接模式最贴切的现实类比是现代综合医院的就医流转或企业标准化客服工单流转：

- 患者首先来到**挂号分诊台**（初始 Agent），导诊护士采集患者基本病症与身份信息；
- 确认信息后，系统交接给**专科门诊医生**（次级 Agent），医生调取病历进行针对性问诊并开具检查；
- 若需化验，控制权进一步交接到**检验科**，待报告出具后再交接回门诊。

由此可见，交接模式最适用于：

- **分阶段收集信息的客服流程**：必须一步步引导用户提供必要参数（如订单号、故障表现、联系方式）；
- **多阶段对话体验**：用户在不同生命周期需要与不同专业风格的对话界面交互；
- **存在顺序约束（Sequential Constraints）的业务场景**：**后续的高阶能力与工具必须在满足特定的前置条件（Preconditions）之后才被正式解锁**（例如：必须先完成鉴权与信息核验，才能激活退款或修改配置等敏感工具）。

## 架构权衡与关键代价

交接模式带来了极致自然的阶段过渡，但也引入了架构上的复杂度：

- **核心代价**：相比其他多智能体模式，交接模式具有更强的“有状态（Stateful）”特征，这要求架构师必须建立极其严谨的状态管理（State Management）机制，妥善处理跳转冲突、意外断流重试以及回溯等边界情况。
- **架构收益**：**这种高度有状态的设计换来的是多轮对话中上下文的自然延续**，彻底消除了用户在跨领域、跨阶段流转时需要反复重复前置背景信息的割裂感。

## 3.5 Router：并行分发与综合

路由器模式（Router）是一种经典的“分类-分发-聚合”架构，其设计动机在于以单次请求为核心，高效应对涉及多个独立垂直领域的复杂复合查询。在该模式中，系统先设立一个专门的路由步骤对输入问题进行分类判断，随后定向导向专门的子智能体执行查询，并以并行执行结合最终综合（Synthesis）的方式输出连贯结果。

### 工作原理与执行逻辑

路由器模式的核心执行链路清晰而高度模块化，通常由三个关键动作串联而成：

1. **查询分解（Query Decomposition）**：路由器接收到用户的输入后，首先解析其意图，将宏观问题拆解为针对不同知识范畴的子查询；
2. **并行分发（Parallel Dispatch）**：路由器根据分类结果，并发调用零个、一个或多个专门智能体（Specialized Agents）；
3. **结果综合（Synthesis）**：收集各个分支返回的异构数据或解答片段，由路由器将其综合、去重并提炼成一份条理清晰的连贯回答返回给用户。

从工程状态性来看，**路由器模式在设计上通常是纯无状态（Stateless）的，每一个到来的用户请求都被视作完全独立的事件进行孤立处理。**路由器不会默认记忆上一轮的交互细节，只专注于当下输入的极致解析与高效分发。

### 典型适用场景与现实类比

路由器模式的运作机制非常类似于现代企业级综合咨询门户或情报智囊团：

- 当管理者向总机秘书（Router）提问：“请分析我们上一季度华东区产品的营收表现以及当前供应链的仓储周转压力。”
- 秘书不会独自硬抗，而是快速将问题拆分为财务与物流两部分，**同时并行下发**给“财务数据专员”与“供应链专员”；
- 两位专家并行调取各自数据库并产出专报，最后由秘书合并汇总，输出一份结构统一的综合简报。

因此，路由器模式最适用于以下场景：

- **拥有清晰独立垂域（Distinct Verticals）的系统**：知识库或业务领域边界分明，彼此解耦互不重叠；
- **需跨多源并行查询的场景**：依赖并发 I/O 缩短端到端响应时间；
- **必须综合多个智能体结果的复杂检索任务**：例如大型企业级知识库（Enterprise Knowledge Bases）和横跨多业务线的综合多垂域客服助手（Multi-vertical Customer Support Assistants）。

### 架构权衡与优化策略

路由器模式展现了极高的处理效率，但也伴随着明确的工程妥协：

- **核心代价**：无状态设计确保了系统具有出色的横向扩展能力以及每个独立请求的稳定表现；但其致命短板在于对上下文对话历史的钝感。一旦业务要求展开连续的多轮深入追问，系统在每一轮都需要重新付出完整的语义分类与路由调度开销。
- **工程缓解之道**：**在实际系统架构中，最常用的缓解方案是将无状态的 Router 整体打包封装为一项标准工具（Tool），并将其挂载在一个有状态对话智能体（Stateful Conversational Agent）之下。**这样既能由外层有状态智能体维系多轮对话的上下文记忆，又能在需要跨库检索时充分激活内层 Router 的高并发综合优势。

### 四种多 Agent 架构模式汇总对比

为便于在系统设计时进行架构选型，现将 LangChain 阐述的四种核心多智能体模式总结对比如下：

| 模式 | 工作原理 | 最适用于 | 关键代价 |
| --- | --- | --- | --- |
| **子智能体模式（Subagents）** | 主控者（Supervisor）将专门子智能体当作工具调用；主智能体保留对话上下文，子智能体保持无状态实现强上下文隔离；支持并发调用。 | 多独立领域且需要集中控制的场景（如协调日历/邮件/CRM 的个人助理、分派领域专家的研究系统）。 | 每次交互多产生一次模型调用，带来额外的延迟与 Token 消耗。 |
| **技能模式（Skills）** | 按需动态加载专门系统提示词与知识（三层阶梯：名称与描述 → 完整上下文 → 内部文件按需再发现）；技术本质上为单 Agent，属于“准多 Agent 架构”。 | 单智能体多重专门化、支持多团队各自独立维护自身技能资产的场景（如全能编码 Agent、创意工作流助手）。 | 技能一旦加载后会在单 Agent 上下文中持续累积，导致后续交互轮次的 Token 开销明显膨胀。 |
| **交接模式（Handoffs）** | 活跃智能体随上下文动态流转，各智能体均可通过工具调用交接控制权；通过更新状态决定下一个被激活的智能体（切换 Agent 或重置提示词与工具）；状态跨轮持久存活，支撑顺序工作流。 | 分阶段收集信息的客服流程、多阶段复杂对话体验，以及能力需严格满足前置条件后方可解锁的顺序约束场景。 | 相比其他模式状态性更强，需要极其细致的状态管理机制；但换取了多轮跨阶段交互中上下文的自然无缝延续。 |

| 模式 | 工作原理 | 最适用于 | 关键代价 |
| --- | --- | --- | --- |
| **路由器模式（Router）** | 前置路由步骤先对输入进行分类与查询分解，并行调用零个或多个专门智能体执行查询，并将各路结果综合为连贯回答；典型设计为无状态，单请求独立处理。 | 具备独立垂直知识领域、需要跨多个异构数据源进行并发检索，以及必须综合汇总多智能体分析结果的场景（如企业知识库、多垂域客服助手）。 | 无状态设计保障了单次请求的确定性与高稳定性，但面临连续多轮历史追问时会反复承担路由开销（可通过将 Router 包装为有状态对话 Agent 下属的工具来缓解）。 |

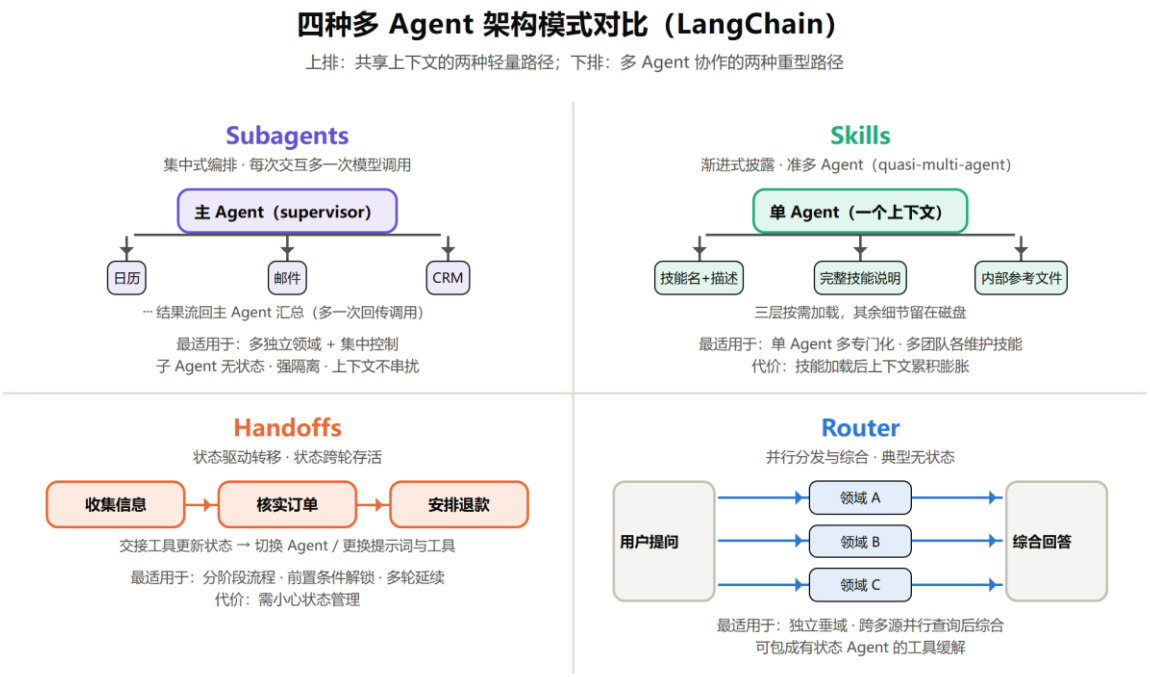



图 6：四种多 Agent 架构模式对比

## 3.6 需求-模式匹配：两张决策表

在明确了子智能体（Subagents）、技能（Skills）、移交（Handoffs）与路由（Router）四种核心模式的底层机制后，架构师面临的真正挑战在于：**面对具体、繁杂甚至充满妥协的业务场景，究竟该如何选择最合适的架构模式?**

LangChain 官方通过大量产业落地实践，提炼出了两张极具工程指导意义的选型决策表。第一张是面向业务特征的“需求-模式匹配表”，帮助我们快速锚定候选架构；第二张是面向工程维度的“能力评分表”，用于对候选架构进行技术可行性与工程约束校验。

### 1. 业务需求与推荐模式速查

在立项或重构初期，业务架构师应当首先审视核心业务形态，根据任务领域边界与执行流特征直接对照下表：

| Your requirements | Pattern |
| --- | --- |

| Your requirements | Pattern |
|---|---|
| Multiple distinct domains (calendar, email, CRM), need parallel execution | **Subagents** |
| Single agent with many possible specializations, lightweight composition | **Skills** |
| Sequential workflow with state transitions, agent converses with user throughout | **Handoffs** |
| Distinct verticals, query multiple sources in parallel and synthesize results | **Router** |

## 2. 架构能力四维评分表

业务诉求只是表象，系统落地的成败往往取决于非功能性需求（Non-functional requirements）与团队组织结构的适配度。下表从工程实现的四个核心维度对四种模式进行了能力定级：

| Pattern | Distributed development | Parallelization | Multi-hop | Direct user interaction |
|---|---|---|---|---|
| Subagents | ⭐⭐⭐⭐⭐ | ⭐⭐⭐⭐⭐ | ⭐⭐⭐⭐⭐ | ⭐ |
| Skills | ⭐⭐⭐⭐⭐ | ⭐⭐⭐ | ⭐⭐⭐⭐⭐ | ⭐⭐⭐⭐⭐ |
| Handoffs | — | — | ⭐⭐⭐⭐⭐ | ⭐⭐⭐⭐⭐ |
| Router | ⭐⭐⭐ | ⭐⭐⭐⭐⭐ | — | ⭐⭐⭐ |

## 怎么读这两张表：四个核心维度的工程内涵

很多工程师在阅读能力评分表时，容易把星级高低简单等同于“架构优劣”，这是极其危险的误区。**星级代表的是模式对某种工程特性的原生支持力度，而非绝对的技术先进性。** 理解这四个维度的真正含义，是做出理性技术选型的前提：

1. **分布式开发（Distributed development）：不同团队能否独立维护组件?**
   - **工程本质：** 康威定律在 Agent 架构中的投射。在大型企业级应用中，负责 CRM 的团队与负责日历的团队不可能每天同步代码。
   - **评分透视：** Subagents 与 Skills 均获得满星（⭐⭐⭐⭐⭐），因为它们支持彻底的物理与逻辑解耦。子 Agent 可以作为独立微服务部署，Skill 可以作为独立包进行版本化管理与热插拔。而 Handoffs 为“—”（完全不支持），因为转移逻辑依赖全局状态机或全局提示词（System Prompt），任何一个 Agent 行为的变动都可能导致转移死循环，多团队协作时极易产生跨域冲突。
2. **并行化（Parallelization）：多个 Agent 能否并发执行?**
   - **工程本质：** 系统应对高耗时、宽扇出（Fan-out）查询时的吞吐与延迟（Latency）瓶颈。
   - **评分透视：** Subagents 与 Router 原生支持并行（⭐⭐⭐⭐⭐）。主 Agent 或 Router 可以一次性生成多个并行调用，同时向多个垂直领域派发请求，等待结果合并。相反，Handoffs 明确标为“—”，因为移交本质上是控制权的**单向接力棒传递**，系统在任意时间点有且仅有一个活动状态持有者，天然无法做到多域并发查询。

3. **多跳（Multi-hop）：该模式是否支持串行调用多个子 Agent?**
   - **工程本质：** 应对多步骤、深度逻辑依赖任务的拓扑表达能力。
   - **评分透视：** Subagents、Skills 和 Handoffs 均具备极高的多跳能力（⭐ ⭐ ⭐ ⭐ ⭐）。它们能够支撑“先查用户画像 -> 再查历史订单 -> 最后生成退款方案”的链式流转。而 Router 在此处评分为“—”，因为标准 Router 采用经典的“扁平分发-收敛”拓扑，设计初衷是单次命中一个或多个目标并返回，自身并不维护深度递归的执行图。
4. **直接用户交互（Direct user interaction）：子 Agent 能否直接与用户对话?**
   - **工程本质：** 人机交互的沟通链路厚度与体验连续性。
   - **评分透视：** Handoffs 与 Skills 获得了满星（⭐ ⭐ ⭐ ⭐ ⭐）。在 Handoffs 中，一旦控制权移交，目标 Agent 就会接管输入输出通道，用户感知到的就是与该垂类专家的直接对话；Skills 模式下本就是单 Agent 全程陪伴。相比之下，Subagents 仅得一星（⭐），因为子 Agent 处于沙箱之内，其产出必须回传给主编排 Agent 经过审查、润色后才能输出给用户，用户无法越过主 Agent 直接干预子 Agent 的内部执行。

### 工程选型双步法：如何运用两张表完成决策

在实际架构设计中，建议团队严格执行以下双步选型流程：

- **第一步：查“需求表”锁定候选模式。** 从业务形态切入，辨析业务究竟是“多垂域并行聚合”（锁定 Subagents / Router），还是“长流程渐进式客服”（锁定 Handoffs），抑或是“工具集众多但主体统一的智能助手”（锁定 Skills）。
- **第二步：查“评分表”做硬约束反向过滤。** 拿团队现实约束去对齐四个维度。例如：业务虽然看起来像多轮客服，但后端涉及 4 个业务线的异构微服务且各团队有独立迭代周期，此时 Handoffs 的“分布式开发能力缺失”将成为致命伤，架构师应果断放弃 Handoffs，转而采用隔离性更好的 Subagents 架构，即便牺牲掉一部分直接交互体验也在所不惜。**选型从来不是挑选能力最多的模式，而是用业务容忍度去交换系统稳定性。**

---

## 3.7 性能实测：三种场景对比

在多 Agent 系统的落地过程中，架构模式的选择绝不仅仅关乎代码组织是否优雅，它直接决定了系统的**推理延迟（Latency）、Token 资金成本（Cost）以及最终的用户交互体验（User Experience）**。

为了量化不同模式在真实运行环境中的工程消耗，LangChain 针对工业界最典型的三种工作负载设计了严格的基准测试。以下是基于实际运行数据的深入拆解。

### 3.7.1 场景一：一次性请求

#### *1. 场景设定*

用户发出一条明确且孤立的单步指令："帮我买杯咖啡（buy coffee）"。系统中存在一个掌握 `buy_coffee` 工具调用的专门化 Agent。

#### *2. 实测数据表*

| Pattern | Model calls | Notes |
|---|---|---|
| Subagents | 4 | Results flow back through main agent |
| Skills | 3 | Direct execution |
| Handoffs | 3 | Direct execution |
| Router | 3 | Direct execution |

#### *3. 为什么：底层调用链路剖析*

为什么同构的任务，Subagents 比其他三种模式多出整整 1 次模型调用（Model calls）？

- **直接执行模式（Skills、Handoffs、Router）：**
  - 第 1 次调用：入口判断意图并分派/加载工具；
  - 第 2 次调用：专职 Agent 决定触发 `buy_coffee` 工具；
  - 工具在本地代码环境中执行并返回物理结果；
  - 第 3 次调用：专职 Agent 消化工具执行结果，直接组织自然语言回复用户。总调用数锁定为 3 次。
- **集中管控模式（Subagents）：**
  - 主 Agent 接收请求，第 1 次调用决定将子任务委托给"咖啡订购子 Agent"；
  - 子 Agent 被拉起，第 2 次调用触发 `buy_coffee` 工具；
  - 工具执行完毕，子 Agent 进行第 3 次调用生成执行摘要，并将结果**回传（Flow back）**给主 Agent；
  - 主 Agent 作为顶层交互唯一出口，必须发起第 4 次模型调用，审查子 Agent 的执行结果并输出最终给用户的答复。

#### *4. 工程启示*

**在单任务、一次性请求场景下，Handoffs、Skills 和 Router 的调用效率最高，均为 3 次调用；Subagents 必然引入额外的集中管控开销（Overhead）。** 这个额外的第 4 次调用不是技术缺陷，而是为了实现全局安全审计、统一输出格式与集中控制所必须支付的架构代价。对于低客单价、低延迟容忍度的单次任务，切忌无脑引入集中式 Subagents 架构。

### 3.7.2 场景二：重复请求

#### *1. 场景设定*

在同一个会话生命周期内，用户发起两次连续相同维度的请求：

- **Turn 1（第一轮）：** “buy coffee”
- **Turn 2（第二轮）：** “buy coffee again”（再帮我买一杯）

#### *2. 实测数据表*

| Pattern | Turn 2 calls | Total calls | Efficiency gain |
|---|---|---|---|
| Subagents | 4 | 8 | — |
| Skills | 2 | 5 | 40% |
| Handoffs | 2 | 5 | 40% |
| Router | 3 | 6 | 25% |

#### *3. 为什么：上下文继承与状态机差异*

这一场景直观展现了**有状态模式（Stateful patterns）**与**无状态设计（Stateless design）**的鸿沟：

- **Handoffs 与 Skills（效率提升 40%）：** 在 Turn 1 结束时，系统状态并没有被销毁。Handoffs 的当前活动执行者停留在咖啡订购 Agent 上；Skills 已经在当前上下文环境中完整激活了咖啡技能。进入 Turn 2 时，系统无需重新经历“意图路由/任务分配”的初始阶段，只需经过“理解上下文直接下发工具”（第 1 次）与“根据工具输出给出应答”（第 2 次）即可。第二轮仅消耗 2 次调用，两轮累计调用 5 次。
- **Router（效率提升 25%）：** Router 通常不强行持有专职 Agent 的活跃状态，Turn 2 依然需要 Router 快速研判并直接定向至旧通道，因此 Turn 2 耗费 3 次调用，总计 6 次。
- **Subagents（效率提升 0%）：** 采用纯粹的无状态编排设计。每一次对话轮次对主 Agent 而言都是全新的任务委托。主 Agent 不会在全局记忆中假定子 Agent 依然就绪，仍然严格走完“委派 -> 子 Agent 决策 -> 工具回传 -> 主 Agent 包装”的全量 4 次调用，两轮累计达到 8 次调用。

#### *4. 工程启示*

**有状态模式（Handoffs、Skills）凭借上下文延续能力，在连续重复交互中能带来高达 40%–50% 的调用量与响应延迟收益；Subagents 则以固定的高昂调用开销，换取了严格的无状态幂等性与上下文隔离（Context isolation）。** 如果你的产品定位是客服坐席或多轮业务办理，状态持久化的架构将为你节省巨额的模型调用费用。

### 3.7.3 场景三：多领域查询

#### *1. 场景设定*

用户发出一条跨越多个垂直领域的综合性复杂请求："对比 Python、JavaScript 和 Rust 在 Web 开发中的表现（Compare Python, JavaScript, and Rust for web development）"。

测试基准设定：系统内包含对应三个语言的专门 Agent，每个 Agent 挂载约 2000 个标记（Tokens）的专业领域文档。底层大模型原生支持并行工具调用（Parallel tool calling）。

#### *2. 实测数据表*

| Pattern | Model calls | Total tokens | Notes |
|---|---|---|---|
| Subagents | 5 | ~9K | Each subagent works in isolation |
| Skills | 3 | ~15K | Context accumulation |
| Handoffs | 7+ | ~14K+ | Sequential execution required |
| Router | 5 | ~9K | Parallel execution |

#### *3. 为什么：并发能力与上下文爆炸的博弈*

该场景是整项测试中最具工程震撼力的一组数据，它彻底揭示了**模型调用次数与 Token 消耗量之间的非正相关性**：

- **Subagents 与 Router（并行吞吐最优，~9K Tokens）：** 这两种模式支持真正的并发扇出。主编排器/路由器利用并发能力，一次性向 3 个独立的语言子 Agent 分派任务。关键在于：**每一个子 Agent 在物理隔离的沙箱中运行，只加载属于自己的 2000 Tokens 文档。** 最终主 Agent 仅汇总各子 Agent 的提炼结论（约 1K 结果上下文），整体消耗精准控制在约 9K Tokens 左右。
- **Skills（调用量最少，但遭遇 Token 膨胀）：** Skills 表面上只用了 3 次模型调用，但代价是灾难性的。由于 Skills 属于单 Agent 上下文，为了完成对比，系统不得不将 Python、JavaScript、Rust 三门语言的文档及工具定义全部注入到同一个会话窗口中。多轮累积导致上下文堆叠达到 ~15K Tokens。**调用次数少绝不等于省钱，上下文累积（Context accumulation）造成的单次推理高单价直接逆转了成本优势。**
- **Handoffs（串行执行陷阱，7+ 次调用，~14K+ Tokens）：** 移交模式暴露了其底层致命短板——**无法并行。**面对多域并发任务，Handoffs 只能串行执行（Sequential execution）：系统先移交给 Python Agent 记录观点，再移交给 JS Agent 补充观点，最后移交给 Rust Agent 整合。链路极长，导致调用次数暴增至 7 次以上，且随着移交链路的拉长，对话历史越积越厚，Token 消耗同样发生恶性膨胀（~14K+）。

*4. 工程启示*

**在涉及大知识库的多领域复杂查询中，具备并行执行与上下文隔离机制的模式（Subagents、Router）具有压倒性的能效优势；Subagents 相比 Skills 能够减少 67% 的整体 Token 消耗，成功遏制了多领域混杂时的 Token 膨胀（Token bloat）。**

---

### 3.7.4 性能总结表与整体解读

为了让性能特征更加直观，LangChain 将上述三个实测场景的架构适应性收敛为一张全局概览表：

| Pattern | Single requests | Repeat requests | Parallel execution | Large-context domains |
|---|---|---|---|---|
| Subagents | — | — | ✅ | ✅ |
| Skills | ✅ | ✅ | — | — |
| Handoffs | ✅ | ✅ | — | — |
| Router | ✅ | — | ✅ | ✅ |

***整体解读：把测试数据翻译为生产环境的工程决策***

测试数据不会撒谎，但工程师必须跳出局部指标的盲区。面对这张总结表，资深架构师应当读出以下四条不可动摇的工程铁律：

1. **没有普适的“最优解”，只有与工作负载特征（Workload characteristics）匹配的妥协解。**

脱离业务负载谈架构优劣毫无意义。如果你的业务 90% 都是瞬时、离散的一次性查询，选择 Subagents 就是对服务器成本和响应延迟的直接浪费；但如果你的业务必须调取海量专业垂直知识库，选择 Skills 或 Handoffs 就会瞬间击穿上下文窗口，带来成倍的模型账单。

2. **区分“调用次数成本”与“Token 吞吐成本”。**

很多团队在技术评审时只盯着“调用了几次模型”，盲目崇拜 Skills 模式“3 次搞定一切”的表象。场景三的实测狠狠击碎了这一错觉：单次调用承载 15K 的海量上下文，其计算成本和首字延迟（TTFT, Time to First Token）远比 5 次各承载 2K 上下文的独立轻量调用昂贵得多。**在长文本时代，上下文隔离带来的 Token 削减价值，远高于节省一两次模型调用。**

3. **控制权与自治性的取舍决定了扩展上限。**

Subagents 之所以在单次和重复请求中表现垫底，是因为它强行保留了“中央控制”的审查流；而它在多领域并行中登顶，也是因为其“强隔离分治”思想切断了错误的跨域污染。当你需要构建跨部门协同、组件高度解耦、防御提示词注入的企业级系统时，必须承受 Subagents 的基础开销。

4. **渐进式演进路径。**

在工程落地时，推荐遵循由轻至重的演进路线：原型验证阶段优先采用轻量的 **Skills** 快速验证能力；当对话具备强流水线属性且强依赖人机对齐时，演进为 **Handoffs**；一旦系统接入的垂类知识库产生严重污染、或者跨域并发查询成为核心负载时，坚决重构成 **Router** 或 **Subagents**。

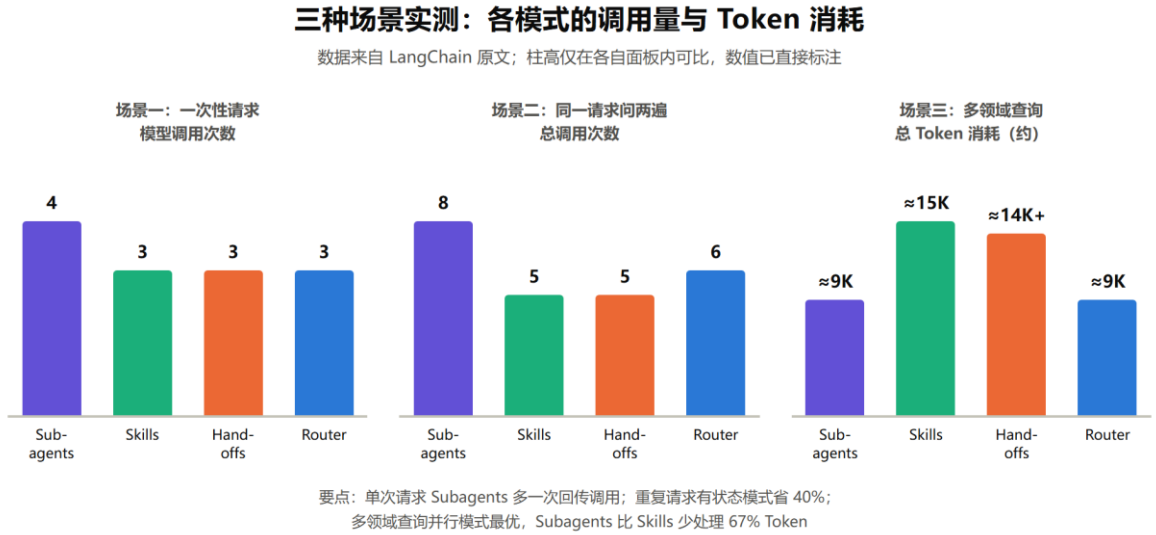



图 7：三种场景下各模式的模型调用量与 Token 消耗对比

# 第 4 章 综合决策框架：两篇文章的合流

## 4.1 五条共识原则

在深入探讨不同架构流派的细节之前，我们必须首先建立底层的架构共识。虽然 Anthropic（文章 A）聚焦于对过度复杂系统的“降维收编”，而 LangChain（文章 B）侧重于面向复杂需求的“模式选型”，但两套方法论在深层工程哲学上表现出了高度一致。教材将其归纳为五条不可动摇的黄金原则：

1. **单智能体（Single Agent）是系统架构的默认起点与基线。**两篇文章一致强调，切勿在项目初始阶段直接引入复杂的多智能体编排。文章 B 明确提出“先单 Agent 与良好提示词工程，先加工具后加 Agent，只有撞到明确能力上限才升级多 Agent”的演进路径；文章 A 则指出，随着现代前沿模型（如 Claude Opus 4.5 起）在单上下文规划、指令遵循及批量工具调用能力的飞跃，在绝大多数企业级场景中，“单循环（Main Loop）+ 渐进式披露”的架构在输出质量上往往超越了碎片化的多智能体系统。**单智能体不仅是开发的起点，更是评估后续一切架构复杂性代价的性能与质量基线。**
2. **上下文窗口（Context Window）是系统中最昂贵的稀缺核心资源。**多智能体系统的一切架构拆分，本质上都是围绕“上下文的分配与隔离”展开的。文章 B 将“上下文管理”列为引入多智能体的第一硬约束，指出无效信息堆积会导致模型注意力分散与推理性能衰减；文章 A 则从反向证明，盲目拆分子智能体会引发上下文强行割裂，带来高昂的有损交接（Lossy Handoffs）与多轮摘要开销。**保持一个干净、高信噪比且连贯的上下文，是决定智能体成败的生命线。**
3. **渐进式披露（Progressive Disclosure）是单智能体与多智能体收敛的交汇点。**两篇文章不约而同地将解决“能力膨胀”的希望寄托于按需加载机制。文章 A 提倡通过技能（Skills）披露业务规则与上下文，通过工具搜索（Tool Search）动态检索工具定义，使得如 Claude Code 等高复杂度工程的系统提示词（System Prompt）削减幅度超过 80%；文章 B 中的 Skills 模式同样依托三层渐进加载机制实现轻量化调度。**与其通过拆分物理 Agent 来隔离上下文，不如在主循环内通过渐进式披露按需注入上下文与工具。**
4. **架构拓扑必须服务于组织边界与实际工作负载特征。**系统架构从不是空中楼阁，它受制于康威定律（Conway's Law）与业务形态。文章 A 警示团队，过度按照组织职能切分子智能体会演变为“弗兰肯斯坦提示词（Frankenprompt）”与口吻割裂的灾难，主张由中央小团队统筹系统提示词与主循环；文章 B 则将“分布式开发（Distributed Development）”作为多智能体架构的第二大现实约束，并通过严密的需求匹配矩阵论证了不同业务负载（如多跳推理、并行化任务）对拓扑结构的客观要求。**架构的选择既要契合开发团队的协作边界，更要契合请求负载的计算图结构。**
5. **一切架构演进必须完全由度量指标与对照实验决定。**架构决策严禁依赖直觉或“炫技式设计”。文章 A 强调“评测（Evaluation / Eval）先行”，主张在重构前建立包含至少 10 条黄金样本的评测集，强调

“一次只改一件事”，并在监控中高度关注“任务翻转（Task Flips）”而非仅看宏观总分；文章 B 则直接以三场景基准测试的实测数据（模型调用次数、Token 消耗量、端到端延迟）为依据，量化各模式的适用边界。**没有量化评测支撑的架构升级，本质上是对系统可维护性的无序消耗。**

---

## 4.2 视角差异与互补

理解两篇文章的关系，关键在于看清作者所处的工业视角：**文章 A 是在为已经陷入多智能体泥潭的企业做“减法”，教你如何安全收编与降级；文章 B 是在为准备跨越单智能体边界的团队做“加法”，教你如何科学选型与扩展。**

| 比较维度 | 文章 A（Anthropic 视角：做减法与收编） | 文章 B（LangChain 视角：做加法与选型） | 教材视角的互补融合 |
|---|---|---|---|
| **出发点与背景** | 企业内部子智能体蔓延（Subagent Sprawl），面临延迟翻倍、上下文碎片化、局部评测失效等故障模式。 | 单智能体能力遭遇物理瓶颈，面临上下文超载或跨团队分布式协作开发受阻。 | 覆盖了系统全生命周期：从起步选型（B）到系统治理与恶性复杂度重构（A）。 |
| **对多 Agent 的态度** | 警惕与克制。指出企业默认过度使用多 Agent，主张收编回单循环。 | 客观且场景化。肯定多 Agent 在特定负载下的价值（如研究系统实测收益达 90.2%）。 | 视多 Agent 为“昂贵的专用武器”：严控准入门槛，仅在特定强收益场景部署。 |
| **对 Skills 的定位** | 主循环内的“容器”。用来收纳业务步骤与上下文规则，降维替代子智能体。 | 准多智能体（Quasi-Multi-Agent）架构之一。位于单 Agent 与重型多 Agent 之间的过渡态。 | 统一将 Skills 定位为“上下文与能力的模块化载体”，优先作为单循环的动态插件。 |
| **延迟与成本量化** | 聚焦执行效率：指出单循环比多 Agent 框架通常约快 2 倍，规避了嵌套循环等待。 | 聚焦 Token 负载：实测多领域查询下 Subagents 比 Skills 节省 67% Token，但多 1 次模型调用。 | 揭示了核心工程权衡：单循环省调用轮次与延迟；隔离型多 Agent 在海量领域知识下省 Token。 |
| **读者的场景映射** | **适合阅读状态**：系统维护困难、响应极其缓慢、口吻不一、Prompt 难以调试的存量项目。 | **适合阅读状态**：正面临工具库过大、需要高并发深度研究、或需多团队独立交付的新项目。 | 读者应根据当前系统的技术债水平与业务负载特征，交叉采用两套工具箱。 |

### 光谱统一：看似矛盾，实则是同一个工程光谱

两篇文章看似在多智能体取舍上互有交锋，但若将 LangChain 提出的四种模式放入统一的坐标系中，我们会发现它们实际上分布在一个以“上下文与控制权隔离度”为轴的连续工程光谱上：

```
[重度多 Agent] ------------------------------------------------------> [轻度多 Agent / 增强单 Agent]
Subagents 模式  -->  Router 模式  -->  Handoffs 模式  -->  Skills 模式  -->  原生单 Agent 主循环
(强隔离/无状态)     (无状态分发)       (状态交接转移)       (三层渐进披露)     (单一模型/单一上下文)
```

在这个光谱中：

- **重度端（Subagents / Router）**：以增加额外编排调用（如 Supervisor 或 Dispatcher 调用）为代价，换取极高的上下文纯净度与强物理隔离。
- **轻度端（Skills / 单循环）**：以单上下文承载信息为特征，利用渐进式披露控制 Token 膨胀，换取最低的系统调用轮次和更直接的端到端交互。

**Anthropic 的核心论点并非彻底否定光谱左端，而是主张将行业的工程重心整体向右移动。**在早期模型推理能力受限时，工程师被迫过早滑向左端的 Subagents 架构；而在现代前沿模型的强大能力下，绝大多数业务场景在光谱右端的"单 Agent + Skills / 工具搜索"即可获得最优的性能与延迟收益。只有在遇到特定工作负载极限时，系统才需要向左侧试探性延伸。

---

## 4.3 架构选型决策树

基于上述光谱与实证数据，我们将两套方法论合流，梳理出面向现代 Agent 工程的标准化决策路径。

### 阶段一：筑牢单智能体基座

所有业务必须从**单 Agent 主循环**起步。

1. 为系统配备单一强推理模型、高质量系统提示词、清晰的工具定义与完善的端到端评测集（至少 10 条黄金样本）。
2. 在主循环中建立平台安全护栏，包括写权限门（Write Gates）与轮次/Token 成本预算。
3. 遵循工具设计规范：**将工具按用户意图分组，确保全局暴露的工具定义数量严格控制在 30–50 个以内**；前置校验与权限检查内置于工具执行逻辑中，通过结构化返回值向模型提供自我修正的指引。

### 阶段二：应用渐进式披露（遏制过早分化）

当单智能体出现上下文体积膨胀、或工具数量超标导致模型选择命中率退化时，**坚决不要直接拆分子智能体，而应优先采用渐进式披露机制**：

- 若业务包含大量复杂的领域规则、多步骤操作 SOP：将其打包为 **Skills**，利用三层渐进机制（元数据感知 → 规则加载 → 文件/脚本执行）按需装载。
- 若工具总数超过容量上限：引入**工具搜索（Tool Search）**，使主模型仅在需要时检索并动态挂载具体工具的 Schema。

### 阶段三：边界撞墙时的多模式跃迁

只有在单智能体结合渐进式披露后，依然撞上明确的不可调和上限时，方可沿着决策树向多智能体拓扑跃迁：

- **分支 A（并行化与上下文丢弃诉求）**：当且仅当系统需要并行探索独立研究路径、处理可能污染主线程的海量非结构化文本、或需要独立的验证沙箱时，跃迁至 **Subagents 模式**或 **Router 模式。**实测证明，在多领域并行查询下，Subagents 能够比 Skills 节省高达 67% 的 Token 消耗，且 Anthropic 的深度研究架构在并行推理场景下相较单模型取得了 90.2% 的显著增益。
- **分支 B（流程顺序与能力权限隔离诉求）**：当业务具备严格的阶段性准入条件、需要由浅入深的对话式信息收集、或明确的状态机生命周期时，跃迁至 **Handoffs 模式。**利用交接工具显式切换当前处于激活状态的专门 Agent，实现跨轮次的状态保留与控制权转移。
- **分支 C（多团队分布式开发约束）**：当架构瓶颈源于组织协同而非技术本身时，采取分层治理架构——**中央核心团队持有全局系统提示词、主循环编排与端到端评测套件；各业务线独立开发并交付标准化的 Skills、MCP 工具或黑盒 Subagents。**

### 阶段四：存量过度编排系统的收编

若团队已接手了由十几个专家子智能体构成的庞大系统，并遭受延迟恶化与拼凑提示词的困扰，应按文章 A 的成熟路径启动系统收编：

1. 先行建立端到端评测集与打分细则（Rubric）。
2. 优先收编相互依赖深、共享数据源、常被连续调用的专家 Agent。
3. 将子智能体降级为工具，或利用官方迁移套件彻底折叠为 Skills + MCP 工具。

**必须深刻铭记：决策树是指导工程实践的地图，而不是僵化的教条法律；架构拓扑上的每一次跃迁，都必须以严格的评测对比数据为通行证。**

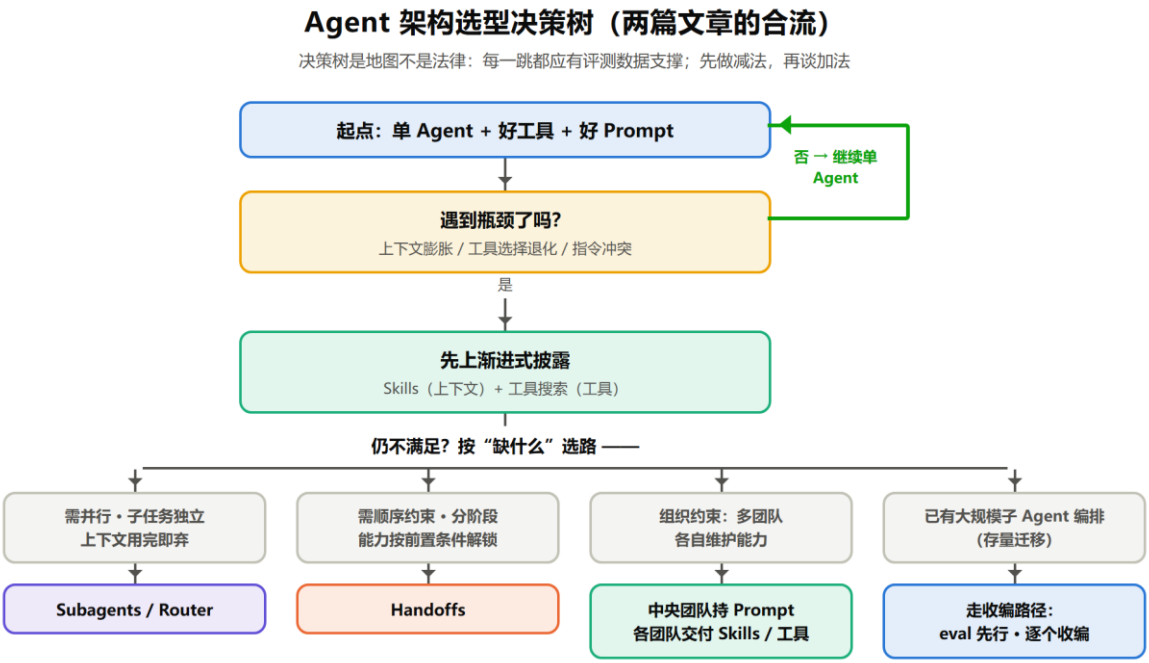



图 8：Agent 架构选型决策树

## 4.4 实践检查清单

### A. 新项目起步清单

- [ ] **单循环优先**：项目是否以单一模型、单上下文主循环作为架构的默认第一起点？
- [ ] **意图分组**：全局工具是否已按业务领域与用户意图进行了逻辑分组，避免平铺暴露？
- [ ] **工具量阈值**：系统提示词中默认常驻的工具定义总数是否严格控制在 30–50 个以内？
- [ ] **内置安全门**：涉及写操作、资金转移或高危外部修改的工具，是否均已配置前置状态检查与写权限门（Write Gates）？
- [ ] **资源硬预算**：执行 Harness 中是否已强制注入单次调用的最大轮次阈值、Token 消耗预算与全局超时熔断机制？
- [ ] **知识库外置**：偶尔查阅的参考手册、长文本规范是否已彻底剥离出系统提示词，转为搜索工具或按需读取的文件？

### B. 打算上多 Agent 前自查清单

- [ ] **瓶颈客观性**：系统是否真的出现了上下文窗口不可逆膨胀导致的推理退化，或跨团队代码合并冲突的组织级硬约束？
- [ ] **穷尽渐进披露**：在决定拆分 Agent 之前，是否已在单循环内尝试过通过 Skills 延迟加载规则，或通过工具搜索动态提供 Schema？

- [ ] **任务边界紧凑性**：计划交由子智能体承接的任务，是否满足“依赖主系统上下文极少、且最终仅需向主循环回传紧凑提炼结果”的强隔离特征？
- [ ] **架构极简原则**：如果确实需要多智能体协作，是否选择了能满足当前负载的最简拓扑（优先考虑简单的 Router 或 Handoffs，而非任意拓扑嵌套的复杂图网络）？
- [ ] **延迟容忍评估**：下游业务场景是否能够接受多智能体编排必然引入的额外模型调用与端到端延迟（单循环响应通常快约 2 倍）？

### C. 存量子 Agent 系统迁移清单

- [ ] **端到端基线**：在修改任何代码之前，是否已建立了覆盖核心业务场景、至少包含 10 条高难度黄金样本的端到端评测集？
- [ ] **路由度量监控**：系统是否已开始追踪意图识别与路由准确率（如模型是否正确调用了对应工具或加载了对应 Skill）？
- [ ] **按热度排序收编**：收编重构是否遵循优先级策略，优先从调用频次最高、共享上下文最密切的子智能体开始合并？
- [ ] **单变量渐进收编**：是否严格遵守“一次只改动一个组件”的原则，逐个收编专家 Agent 并与迁移前基线进行严格回归对比？
- [ ] **重视任务翻转**：评估重构效果时，是否穿透了宏观总分，重点排查是否存在微观上的“任务翻转”（例如：总分略微上升，但原本能答对的客服发货日期查询因主模型直接调用工具未咨询专家而失败）？
- [ ] **消除随机性干扰**：在对比评测中，针对存在模型随机波动的核心测试用例，是否确保每个样本均独立运行了多次以确认结果的统计稳健性？

# 第 5 章 总结与思考题

## 5.1 全书要点回顾

经过前四章的系统剖析与推演，我们从工业落地视角全面拆解了现代 Agent 架构的演进逻辑。本节通过下表对全书核心知识体系进行结构化串联：

| 章节与核心主题 | 核心理论与机制 | 关键技术指标与事实证据 | 核心落地原则与警示 |
| --- | --- | --- | --- |
| **第 1 章 引言：架构选型核心问题** | • 单 Agent 与多 Agent 的本质权衡<br>• 系统复杂度、可观测性与业务价值的匹配关系 | • 过早引入多 Agent 导致拓扑复杂化、调试成本呈指数级增长 | • 架构设计的终极目标是交付确定性业务价值，而非追求理论上的分布式拟人化 |
| **第 2 章 Anthropic 精讲：收编子 Agent** | • **三大保留条件**：①单循环上下文膨胀致可测性能退化；②多任务需并行、用完即弃的新鲜上下文；③单 Agent 选工具困难或系统提示词冲突<br>• **五种失败模式**：上下文管理损耗、子 Agent 蔓延、Frankenprompt、无人把关的局部评测、延迟劣势<br>• **单循环+渐进式披露**：一个模型一个上下文窗口，开放式工具循环；首轮写计划随结果修订（计划者即执行者）；顺序与领域流程进 Skills，硬限制与权限进工具<br>• **收编三路线**：合并领域 Agent、子 Agent 作为工具、折叠成 Skills+MCP 工具 | • 单循环较多 Agent 约**快 2 倍**<br>• Opus 4.5 起模型单循环推理与自我修复能力大增<br>• Claude Code 系统提示词削减**超 80%**<br>• 单 Agent 工具定义**超 30–50 个**时选择准确率出现严重退化<br>• 10 条黄金样本起步，基于 rubric 打分与时间戳回放<br>• 配套官方套件：`build-eval`、`prompt-audit`、`hillclimb`、`cost-optimize` 与 `single-agent-loop-kit` | • **Eval 先行**：首个合并前必须建立端到端评测<br>• 严格追踪**任务翻转（Task Flip）**而非笼统胜率<br>• 原则：度量、一次只改一件事、留下经得起检验的架构 |

| 章节与核心主题 | 核心理论与机制 | 关键技术指标与事实证据 | 核心落地原则与警示 |
| --- | --- | --- | --- |
| **第 3 章 LangChain 精讲：多架构选型** | • **两大约束**：上下文管理、分布式团队开发<br>• **四大协作模式**：<br>1. *Subagents*：Supervisor 集中编排，子 Agent 隔离无状态<br>2. *Skills*：三层加载（名称描述 → 完整上下文 → 内部文件），渐进披露<br>3. *Handoffs*：交接工具切上下文/提示词，状态跨轮存活<br>4. *Router*：分类后并行分发再综合，完全无状态 | • **性能实测证据**：<br>- *一次性请求*：Subagents 4 次调用 vs 其余模式 3 次<br>- *重复请求第 2 轮*：Skills/Handoffs 仅 2 次（**省 40% 调用**），Router 3 次（省 25%），Subagents 仍需 4 次<br>- *多领域复杂查询*：Subagents 与 Router 5 次调用消耗 ~9K token；Skills 3 次却膨胀至 ~15K token；Handoffs 7+ 次消耗 ~14K+ token；Subagents 较 Skills **节省 67% token**<br>• Anthropic 极端复杂多 Agent 研究系统准确率比单 Agent 高 **90.2%** | • **单 Agent 优先**：先写好 System Prompt，先加工具再加 Agent<br>• 没有免费的架构：隔离带来调用与路由损耗，聚合带来上下文污染<br>• 只有撞到单循环明确性能上限才允许拓扑升级 |
| **第 4 章 综合决策框架** | • 工业界五条共识（渐进式披露、端到端评测先于架构拆分等）<br>• Anthropic（自顶向下精简）与 LangChain（自底向上拓扑）视角互补<br>• 阶段化架构决策树与三份工程实践清单 | • 研发链路各阶段标准门禁（单 Agent Baseline、上下文退化拐点、并行度测算） | • 避免 Conway 定律反向绑架技术选型，让系统物理架构回归真实数据流与任务流 |

> **一句话记住全书：先单 Agent、渐进披露、并行才拆 Agent、评测定去留。** 在大模型上下文窗口与推理能力跃迁的背景下，所有多 Agent 架构的红利都在向单循环收拢。除非系统面临无法回避的物理并行、上下文不可逆退化或绝对权限隔离，否则过度拆分必然伴随严重的延迟翻倍、上下文损耗与评测黑洞。

---

## 5.2 思考题

### 思考题 1：Conway 定律在本单位的具体表现

你们团队当前的 Agent 系统拓扑结构，是否直接映射了现有研发部门组织架构（如前端组做 UI Agent、风控组做 Audit Agent、数据组做 SQL Agent）？这种多 Agent 划分究竟是业务任务调度的客观物理必然，还是各组为了代码仓库边界与交付 KPI 妥协的产物？

> **提示：** 回顾第 4 章关于康威定律与系统通信开销的论述。评估在业务场景中跨部门 Agent 调用的序列化/反序列化损耗与信息衰减，思考若将其重构成单循环+多 MCP 工具形态，在组织权限与协作流程上会遭遇哪些非技术阻力。

### 思考题 2：子 Agent 三大保留条件在实际业务中的判定基线

在你的具体业务场景中，如何判定系统已真正满足“子 Agent 三大保留条件”中的任意一条，而非开发者在 Prompt 与工具工程层面的设计失职？

> **提示：** 重点设计可量化的门禁指标：如何测出“单循环上下文膨胀致可测性能退化”的 token 临界拐点；如何通过业务 SLA 论证“必须用完即弃的新鲜上下文进行横向物理并行”；当工具总数超过 30–50 个时，是直接拆分 Agent，还是优先引入 Tool Search、按意图动态注册或渐进式披露？

### 思考题 3：五种失败模式各自的检测信号

如果现有系统已滑入多 Agent 陷阱，在没有完备端到端链路追踪的情况下，工程师如何通过日志、计费账单与用户行为监控，快速诊断出“五种失败模式”的具体信号？

> **提示：** 上下文管理损耗（表现为 Supervisor 重复向子 Agent 交代同一背景的 token 暴增）；子 Agent 蔓延（表现为存在大量单月调用极低或逻辑高度重叠的编排节点）；Frankenprompt（单节点系统提示词超过数千行且包含相互制约的规则冲突）；无人把关的局部评测（各子 Agent 单元评测均显示 95%+ 达标，但用户端任务成功率持续低迷）；延迟劣势（端到端耗时通常比单循环基线高出近 2 倍）。

### 思考题 4：为什么“计划者即执行者”是单循环的本质变化？

传统编排架构普遍推崇“独立 Planner 生成全局 Plan，再由 Worker 串行/并行执行”的范式。为什么 Anthropic 在最佳实践中主张单循环内“首轮输出计划，随工具执行结果动态修订”，并断言“计划者即执行者”是底层范式的质变？

> **提示：** 传统解耦架构假设 Planner 能在未接触底层动态工具原始输出前做出完备预判，实际会导致 Plan 僵化或产生频繁返工重规划的上下文开销。单循环赋予同一上下文即时感知环境副作用、实时原位纠偏的能力，消除了规划层与执行层之间的协议序列化损耗。

### 思考题 5：什么 Workload 该选 Router 而不是 Subagents？

在构建企业级问答或多源信息检索系统时，如果用户请求往往需要调取多个独立业务域的数据，什么情况下应该坚决选择无状态的 Router 模式，而不是具备集中控制权的 Subagents（Supervisor）编排？

> **提示：** 对比两者在上下文生命周期与 token 经济学上的差异。无状态 Router 适合各业务域彼此高度独立、无需多轮跨域深层推理协同，且对并发首字延迟有极致要求的场景。若各数据源之间存在依赖反馈与状态纠缠，Router 则无法替代状态保持系统。

### 思考题 6：重复请求场景下为什么有状态模式能节省 40% 的模型调用？

根据 LangChain 的基准评测，在多轮对话的重复请求场景中，第二轮时 Skills 与 Handoffs 模式仅需 2 次模型调用（Subagents 仍需 4 次），两轮累计 5 次对 8 次、约省 40%，其底层状态与上下文演变机制是什么？

> **提示：** 分析 Supervisor 在 Subagents 模式下的固有路由代价：每一轮交互中，Supervisor 都必须额外承担一次用户意图分发调用，子 Agent 处理后还需 Supervisor 综合响应。而 Handoffs 与加载后的 Skills 会将执行上下文与指令驻留在当前激活环境中，第二轮请求可直接命中目标工具链执行并响应。

### 思考题 7：为什么评测体系要看“任务翻转（Task Flip）”而不是粗粒度的总胜率？

在对 Agent 系统进行收编与重构时，为什么两套架构对比的总分或平均胜率（Win Rate）极具欺骗性？在什么场景下，即使新架构平均得分更高，系统也绝不能贸然上线？

> **提示：** 结合客服合并实测案例：单循环收编后整体延迟下降且总准确率可能提升，但微观上可能出现“主 Agent 拿着订单工具直接草率回答，不再调用物流专家”的确定性衰退。若这类回退发生在极高业务风险（如退款、合规审计）的样本上，即便总分提升，由于系统引入了不可控的任务翻转，仍会造成生产事故。

### 思考题 8：中央团队的职责边界与合理规模

在推行“单循环 + Skills/MCP”标准化架构的企业中，负责 Agent 基础设施的中央团队规模该如何控制？他们的核心交付物与管控边界究竟是什么？

> **提示：** 警惕中央团队成为业务落地的研发瓶颈。中央团队不应负责编写各业务域具体的业务 Prompt 与具体工具逻辑，而应定位于“规则与标准制定者”：维护全局端到端评测基准套件、管理底层 Harness（轮次/成本预算与写权限门禁）、统一 MCP 接入标准，以及提供架构折叠迁移工具链（如回放与回归审计工具）。

---

## 5.3 动手实践作业

### 作业一：为实际业务场景撰写一页架构决策记录（ADR）

- **任务目标：** 选取你当前负责或即将启动的复杂 Agent 业务场景，严格运用第 4 章的“综合架构决策树”，推演并撰写一份标准的一页纸架构决策记录（Architecture Decision Record）。
- **实施要求：**
  1. 必须明确业务当前的真实约束（并发 SLA、每请求成本上限、工具定义总数、团队分工边界）。

2. 严禁直接给出结论；必须依次回溯决策树的每一个判定节点，并提供客观事实与测试证据（如：上下文 token 增长曲线、是否存在物理并行必要性、工具冲突率测试数据）。
3. 明确记录所选架构的放弃项（Trade-offs）与潜在退化应对方案。

- **交付物清单：**
  - `ADR-001-[项目名称]-架构选型记录.md`（包含：背景上下文、决策路径证明链、最终架构拓扑图、被否决方案及量化理由、后续性能退化监控阈值）。

---

## 作业二：设计 10 条黄金样本的端到端评测集与打分 Rubric

- **任务目标：** 贯彻“Eval 先行”原则，为你负责的业务场景提炼出第一套具备生产代表性的端到端黄金评测集，并建立无歧义的判分 Rubric。
- **实施要求：**
  1. **样本分布要求**：10 条样本中，至少包含 3 条常规业务单流样本、3 条长尾复杂/潜在工具冲突样本、2 条边界对抗/无解拒答样本、2 条跨领域复合查询样本。
  2. **评测维度定义**：针对每条样本，不得使用笼统的“回答良好”等主观描述，必须包含：输入 payload、基准事实（Ground Truth）、依赖工具调用序列白名单、禁止出现的幻觉与副作用动作。
  3. **Rubric 细则**：制定 1–5 分的分级量化标准，特别注明降级为 1 分的“一票否决项”（如：未经验证触发写操作、越权访问敏感信息）。
- **交付物清单：**
  - `eval_golden_set.jsonl`（包含 10 条黄金样本的标准输入、期望工具链、期望断言输出与元数据）。
  - `scoring_rubric.md`（评测打分执行标准，涵盖语义合规、工具精确匹配、轮次与 token 消耗上限）。

---

## 作业三：双 Agent 系统折叠收编实验与任务翻转对比

- **任务目标：** 选取或搭建一个由“Supervisor + 2 个子 Agent”（例如：客服系统中的订单查询 Agent 与退换货策略 Agent）构成的微型多 Agent 系统，执行“路线三：折叠成 Skills + MCP 工具”，完成收编重构并运行对照评测。

- **实施要求：**
  1. **建立基线（Baseline）**：在原多 Agent 系统上运行作业二中的 10 条黄金样本，记录端到端延迟、总 token 消耗、各节点调用次数与最终得分。
  2. **收编执行**：利用渐进式披露理念，将退换货策略 Agent 的提示词降级解构为 Markdown 格式的 Skill 文档，将其专用操作封装为受限 MCP 工具挂载回主循环，将系统转为单 Agent 循环模式。
  3. **翻转分析（Flip Analysis）**：在重构后的单循环系统上复跑同一套评测集。对比两组数据，除了统计总体延迟与 token 变化外，必须逐条比对输出差异，重点定位是否存在“任务翻转”（即原系统成功但新系统失败的案例），并深入分析其失败是否源于工具选择退化或缺少领域特定约束。
- **交付物清单：**
  - `refactor_experiment_report.md`（实验对比报告：包含重构前后拓扑说明、延迟/Token/调用次数三项核心性能对比表）。
  - `task_flip_matrix.csv`（10 条样本的任务翻转矩阵：标注 Baseline 状态、收编后状态、翻转类型 `None / Positive / Negative` 及根因归因）。
  - `migrated_skill_spec.md`（收编产出的 Skill 规范文档，展示如何将原子 Agent 提示词解构为领域操作规则）。

# 附录 A 术语表

按拼音字母排序。英文列给出原文用词，方便对照两篇原文阅读。

| 术语 | 英文 | 一句话解释 |
| --- | --- | --- |
| 编排器 | orchestrator / supervisor | 集中调度子 Agent 的主控组件：决定派活、收集结果、汇总回复 |
| 采分者效应 | context rot 的俗称之一 | 上下文过长后模型注意力稀释，关键指令与工具被忽略的现象 |
| 抽屉式提示词 | Frankenprompt | 把所有指令一股脑塞进单个系统提示词、指望模型自己挑的反模式 |
| 交付门 | ship gate | 上线前必须通过的评测关口：任务翻转数与回归套件不劣于基线 |
| 端到端评测 | end-to-end eval | 直接度量最终任务成败的评测方式，与只测单个组件相对 |
| 工具搜索 | tool search | 让模型先检索"有哪些工具"再决定调用哪层的渐进式披露机制 |
| 上下文工程 | context engineering | 管理进入模型上下文的一切内容（指令/工具/知识）的工程学科 |
| 上下文窗口 | context window | 模型单次能读取的全部 token 预算，是 Agent 架构的第一约束 |
| 收编 | fold / folding | 把子 Agent 的能力合并回单 Agent 主循环的重构动作（文章 A 主题） |
| 任务翻转 | task flip | 同一任务在两个版本间"原来对、现在错"的回归，评测的核心盯防对象 |
| 任务描述 | task specification | 子 Agent / 技能对外暴露的名称与说明，供主 Agent 决定是否调用 |
| 评分量表 | rubric | 按维度打分的评测标准，用于把主观判断变成可复现的分数 |
| 评测集 | eval set | 一组带预期结果的测试任务，用于回归对比新旧版本 |
| 渐进式披露 | progressive disclosure | 只在需要时才加载说明与细节，三层加载（名字→说明→全文）是典型实现 |
| 交接 | handoff | Agent 之间显式转移控制权与状态的机制，状态跨轮存活 |
| 金样本 | golden examples | 评测集中人工确认过正确答案的标准样本 |

| 术语 | 英文 | 一句话解释 |
| --- | --- | --- |
| 路由器 | router | 把一个请求并行分发给多个领域 Agent、再综合答案的架构模式 |
| 良性循环 | flywheel / feedback loop | 工具越用越准、越准越用的正向反馈，好工具驱动更好输出 |
| 模型上下文协议 | MCP, Model Context Protocol | 以标准协议挂载外部工具与数据源的开放协议 |
| 默认工具 | default tools | 无需检索、始终在上下文里的高频工具集（低频工具走搜索） |
| 内部参考文件 | internal reference files | 技能包内的详细文档，仅在主 Agent 需要时才读入上下文 |
| 前置条件 | precondition | 分阶段流程中解锁下一阶段所需的先决信息（如订单号） |
| 全有全无 | all-or-nothing | 提示词一次性加载全部内容、无按需取舍的粗放状态 |
| 人字拖架构 | Conway's law 产物 | 组织架构投影到 Agent 架构：每团队一个 Agent，由组织图而非任务决定拆分 |
| 思考点 | thinking blocks | 推理模型输出中的思考段落，是 token 开销的重要组成部分 |
| 收益递减 | diminishing returns | 加第二个 Agent 有感、加第十个无感的现象，警示过度拆分 |
| 提示词链 | prompt chain | 同一 Agent 在流程各阶段更换提示词与工具的轻量编排 |
| 无状态 | stateless | 子 Agent 用完即弃、不保留对话历史的运行方式，天然隔离 |
| 写权限 | write access | 对外部系统的写入能力，收编后需要预算与权限门来约束 |
| 系统提示词 | system prompt | 定义 Agent 角色、边界与工作方式的常驻指令 |
| 显示模式 | explicit reasoning mode | 思考被外部化为可读文本的输出模式，token 成本显著更高 |
| 显式推理模式 | extended thinking | 模型先输出推理段再给答案的模式；显式版成本更高 |
| 现状偏差 | status quo bias | 评测中"改动少=安全"的错觉，掩盖任务翻转等真实回归 |
| 相对熵 | KL 散度等评测度量 | 衡量新旧版本输出分布差异的统计量，文章 A 用于量化漂移 |
| 写权限门 | write gate | 收编后约束"何时允许对外部系统写入"的运行时控制 |
| 预算 | budget | Agent 每轮可用的 token / 调用次数上限，防失控膨胀 |
| 知识库 | knowledge base | Agent 可检索的领域资料集合，按需注入而非全量塞入 |

| 术语 | 英文 | 一句话解释 |
| --- | --- | --- |
| 逐层折叠 | fold one at a time | 一次只收编一个子 Agent、评测定去留的迁移纪律 |
| 子 Agent | subagent | 独立上下文、可由主 Agent 调用的专职 Agent，结果流回主循环 |
| 自治循环 | agent loop | 模型自主决定调用工具、观察结果、继续推理直至完成的循环 |
| 组件化 | modularity | 把能力拆成可独立替换的模块（技能/工具）而非独立 Agent |

# 附录 B 原文信息与延伸阅读

## 原文信息

本教材的两篇精读原文如下（访问日期：2026 年 10 月）：

| | 文章 A | 文章 B |
|---|---|---|
| 标题 | Building agents: When to fold subagents into your main agent loop | Choosing the Right Multi-Agent Architecture |
| 发布方 | Anthropic | LangChain |
| 链接 | claude.com/resources/articles/building-agents-when-to-fold-subagents-into-your-main-agent-loop | langchain.com/blog/choosing-the-right-multi-agent-architecture |
| 主题 | 把存量多 Agent 系统收编回单 Agent 主循环 | Subagents / Skills / Handoffs / Router 四种模式选型 |
| 对应章节 | 第 2 章 | 第 3 章 |

> 说明：两篇文章均为网络博客文章，发布方可能随后更新内容；本教材依据 2026 年 10 月抓取的版本写成，文中所有观点与数字以该版本为准。

## 章节与原文的对应关系

| 教材章节 | 内容来源 |
|---|---|
| 第 1 章 引言 | 两篇文章的核心论点对照（编者组织） |
| 2.1–2.3 | 文章 A 开头：为什么要考虑收编、渐进式披露、五种失败模式 |
| 2.4–2.5 | 文章 A：单循环设计、任务完成度评测、三条收编路线 |
| 2.6–2.7 | 文章 A：多团队共建单 Agent、官方工具箱与核心原则 |
| 3.1–3.5 | 文章 B：单 Agent 前提与两大约束、四种模式精讲 |
| 3.6–3.7 | 文章 B：三个性能场景、选型原则 |
| 第 4 章 综合决策框架 | 两篇文章的合流（编者组织，观点均注明出处） |
| 第 5 章 总结与思考题 | 全书收束（编者组织） |

## 延伸阅读

以下条目均在两篇原文中被直接提及或引用，可作为下一步阅读路线：

1. **Anthropic 多 Agent 研究系统**（*How we built our multi-agent research system*，Anthropic Engineering 博客）——第 3 章 90.2% 提升数据的原始出处，详细拆解 Opus 主 Agent + Sonnet 子 Agent 的研究系统架构、编排器设计与并行策略。

2. **Agent Skills 与 Claude Code 文档**（docs.claude.com）——渐进式披露的工程实现：技能的三层加载结构、`/skill-creator` 与 `/skill-doctor` 等官方技能工具的用法。
3. **MCP（Model Context Protocol，modelcontextprotocol.io）**——把外部系统挂载为 Agent 工具的开放协议；第 2 章"工具按用户意图分组""权限放进工具里检查"的落地载体。
4. **文中提到的 Claude API skills 与迁移工具包**（single-agent-loop-kit）——`/claude-api build-eval`、`prompt-audit`、`hillclimb`、`cost-optimize` 四个官方技能的说明，见 Anthropic 官方文档与发布页。
5. **LangChain Deep Agents 文档**（LangChain 官方文档）——Subagents 模式的开箱即用实现，几行代码即可为 Agent 挂载子 Agent。

## 引用与免责说明

- 本教材为教学用途的二次整理：原文观点、案例与数字忠实转述；图表为编者依原文内容重绘，数据图（图 7）的数值来自文章 B 的基准测试章节。
- 交叉对照、章节结构、思考题与作业为编者设计，不代表原文作者立场。
- 如原文后续更新，以原文最新版本为准。

# 来源与说明

· 精读原文一: Anthropic《When to fold subagents into your main agent loop》

· 精读原文二: LangChain《Choosing the Right Multi-Agent Architecture》

· 本材料为教学用途的二次整理; 观点与数字忠实转述原文, 章节编排为编者设计

· 原文后续更新时, 以原文最新版本为准

---

由 FiroYu 的 Agent 整理

https://github.com/FiroYu/learning-vault